

C102「AI・機械学習：ニューラルネットワークによる教師あり
学習」

学生番号：242C2016 氏名：奥村直

知的システム工学科システム制御コース

2025年12月15日

1 目的

本演習の目的は、実際に Matlab のプログラミングを行うことにより、AI・機械学習の基礎の一つである「ニューラルネットワークによる教師あり学習」を理解することである。

2 第1週：最急降下法

2.1 演習課題 1.1

2.1.1 演習課題 (a)

(a) 初期値 w やステップ幅 ep を様々な値に変更し、最急降下法での収束がどのように変わるか確かめよ。初期値は描画範囲（-5～5 の範囲）で 2 刻みくらいで変更する。ステップ幅 ep は 0.8, 0.5, 0.2, 0.1, 0.05, 0.01 などに変更する。必要に応じて、反復回数 $iters$ も変更する。

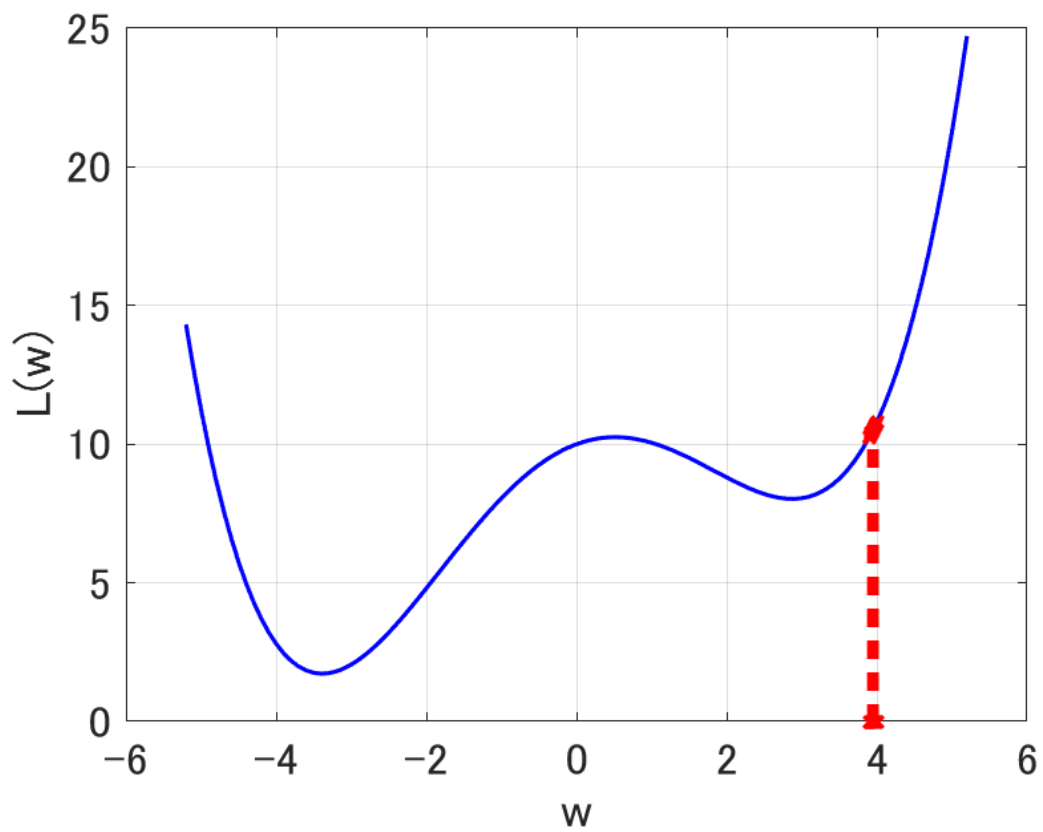


図 2.1: $w=4, ep=0.001, iters=20$ のときのグラフ

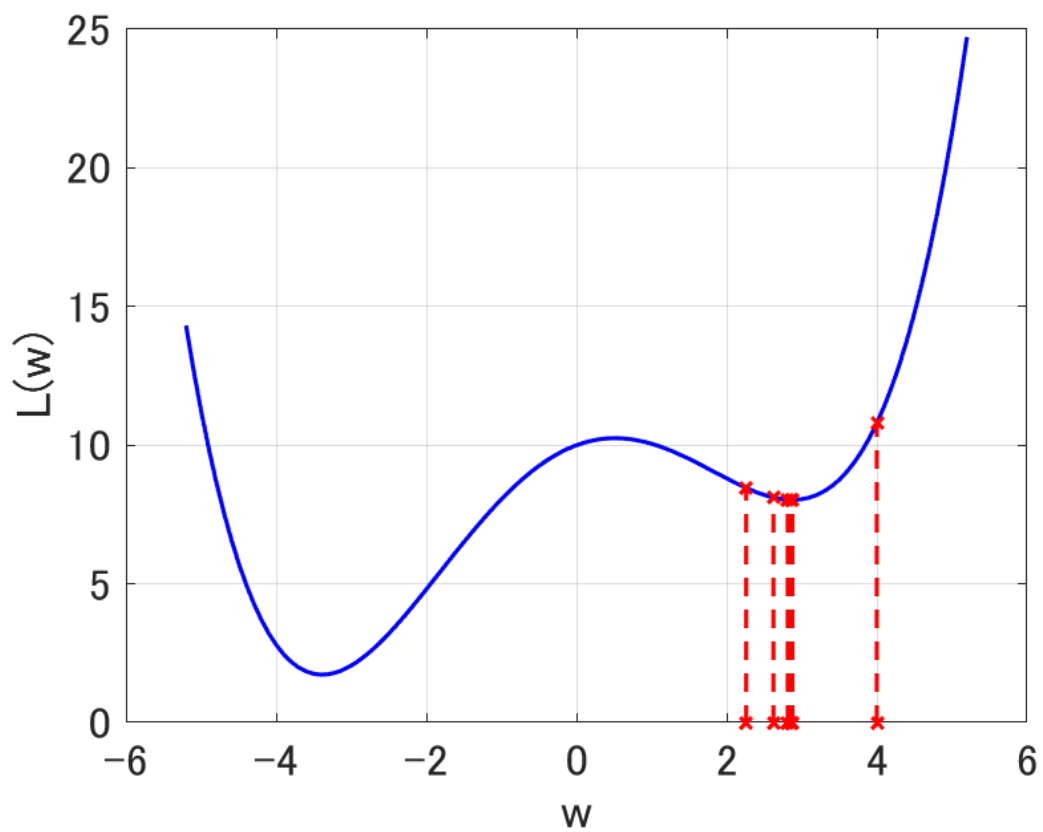


図 2.2: $w=4, ep=0.3, iters=20$ のときのグラフ

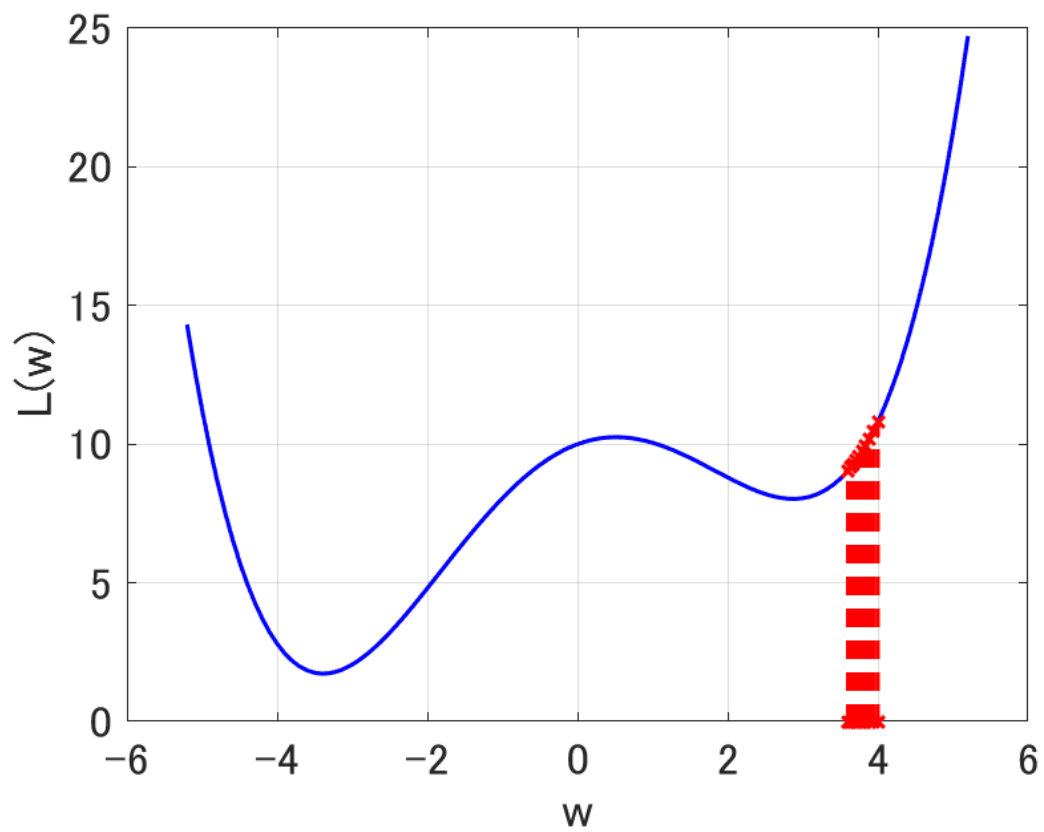


図 2.3: $w=4, ep=0.01, iters=10$ のときのグラフ

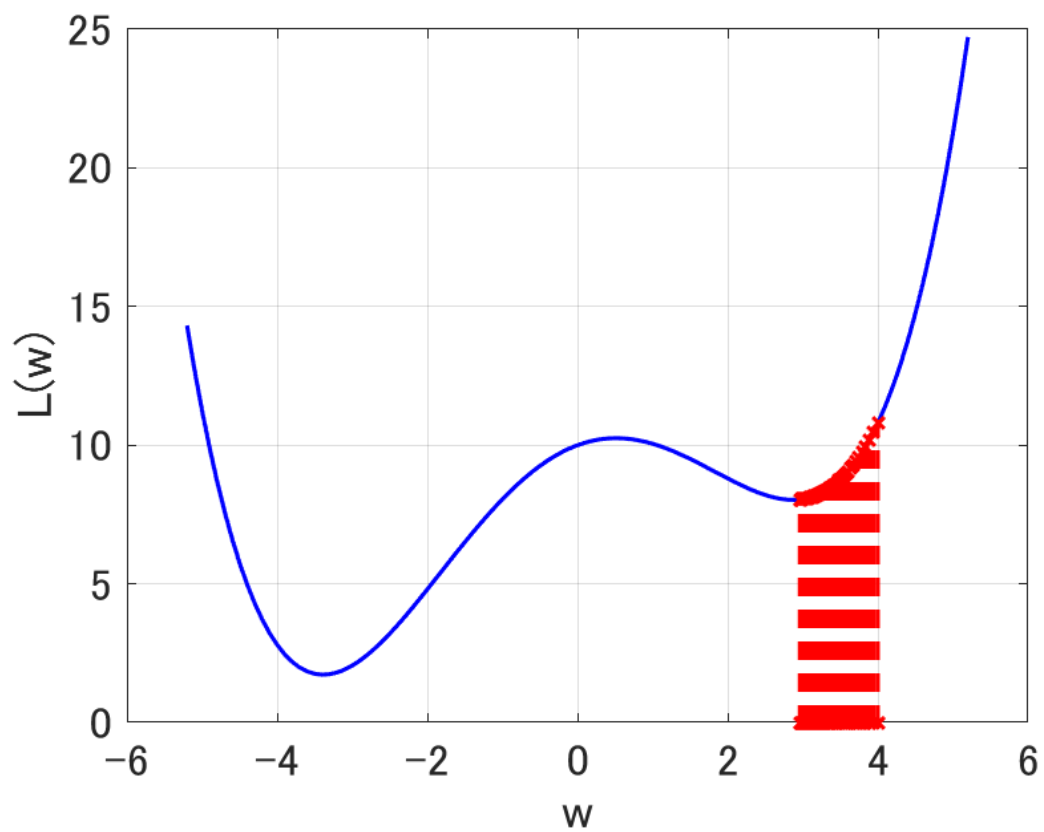


図 2.4: $w=4, ep=0.01, iters=70$ のときのグラフ

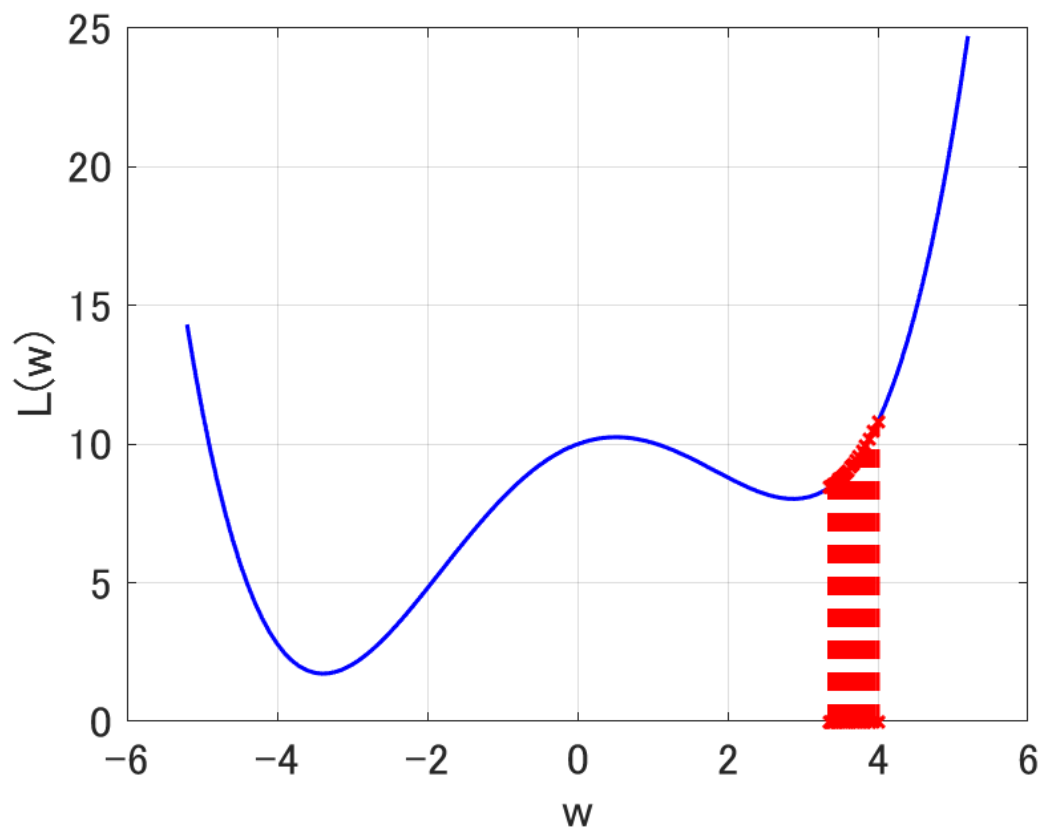


図 2.5: $w=4, ep=0.01, iters=20$ のときのグラフ

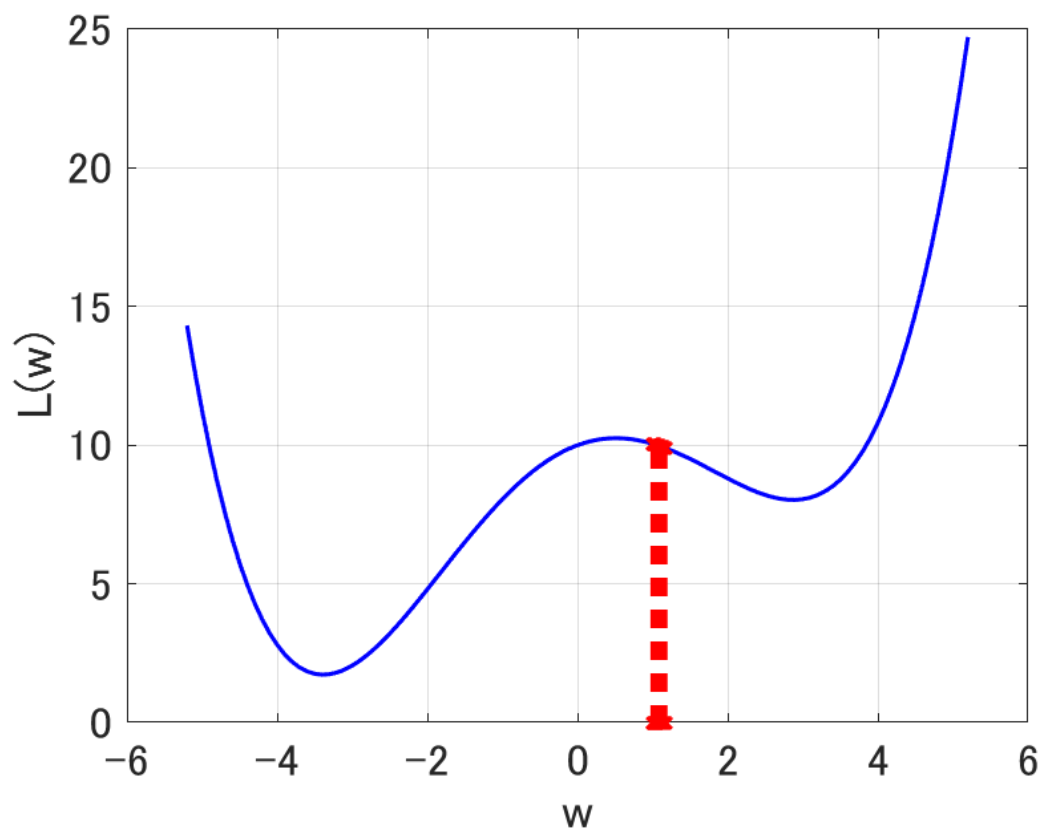


図 2.6: $w=1, ep=0.01, iters=20$ のときのグラフ

2.1.2 演習課題 1.1 (b)

(b) 目的関数が

$$L(w) = \sin(2w) + \frac{w^2}{8} + 2 \quad (2.1)$$

の場合について、プログラムを変更して「saikyu12.m」とし、(a)と同じように様々な初期値とステップ幅に対してどのような振る舞いになるかを確認せよ。この場合の勾配 $\frac{\partial}{\partial w} L(w)$ は自分で計算すること。
目的関数 $L(w)$ の勾配を求めると、

$$\frac{\partial}{\partial w} L(w) = 2 \cos 2w + \frac{w}{4} \quad (2.2)$$

であることが分かるから、saikyu11.m の

```
function g = grad(w)
g = w.^3/5 - 2*w + 1;
end
```

の関数を以下のように変更し、

```
function g = grad(w)
g = 2 * cos(2 * w) + w/4;
end
```

saikyu12.m の全体のソースコードを次に示す。

Listing 2.1: saikyu12

```
1  %%%% 以下を入力 %%%%
2  % プログラム1.1: saikyu11.m
3  %
4  % 変数の目的関数に対する最急降下法1
5  %
6  % すべての変数の値を初期化する。グラフをすべて閉じる。
7  clearvars, close all
8  % ← (A)
9  w = 4;
10 ep = 0.1;
11 iters = 20;
12 ws = w;
13 for k = 1:iters
14     g = grad(w);
15     w = w - ep*g;
16     ws = [ws; w];
17 end
18 %
19 %% 目的関数のグラフを描く
20 % w: 変数. 範囲: から刻みでまでの範囲 -5.20.15.2
21 % Lw: 目的関数の値
22 w = -5.2:0.1:5.2;
23 Lw = obj_func(w);
24 plot(w, Lw, 'b-', 'LineWidth', 1.5), grid on, hold on
25 xlabel('w'), ylabel('L(w)')
26 set(gca, 'FontSize', 16) % フォントサイズをポイントにする. 16
27 % ← (B)
28 for k = 1:iters
29     yk = obj_func(ws(k));
30     plot([ws(k), ws(k)], [0, yk], 'rx--', 'LineWidth', 1.5);
31 end
32 %
```

```

33 %% 目的関数 (objective) の計算function
34 function y = obj_func(w)
35 y = sin(2 * w) + w.^2/8 + 2;
36 end
37 % ← (C)
38 function g = grad(w)
39 g = 2 * cos(2 * w) + w/4;
40 end
41 %%%% 以上を入力 %%%%

```

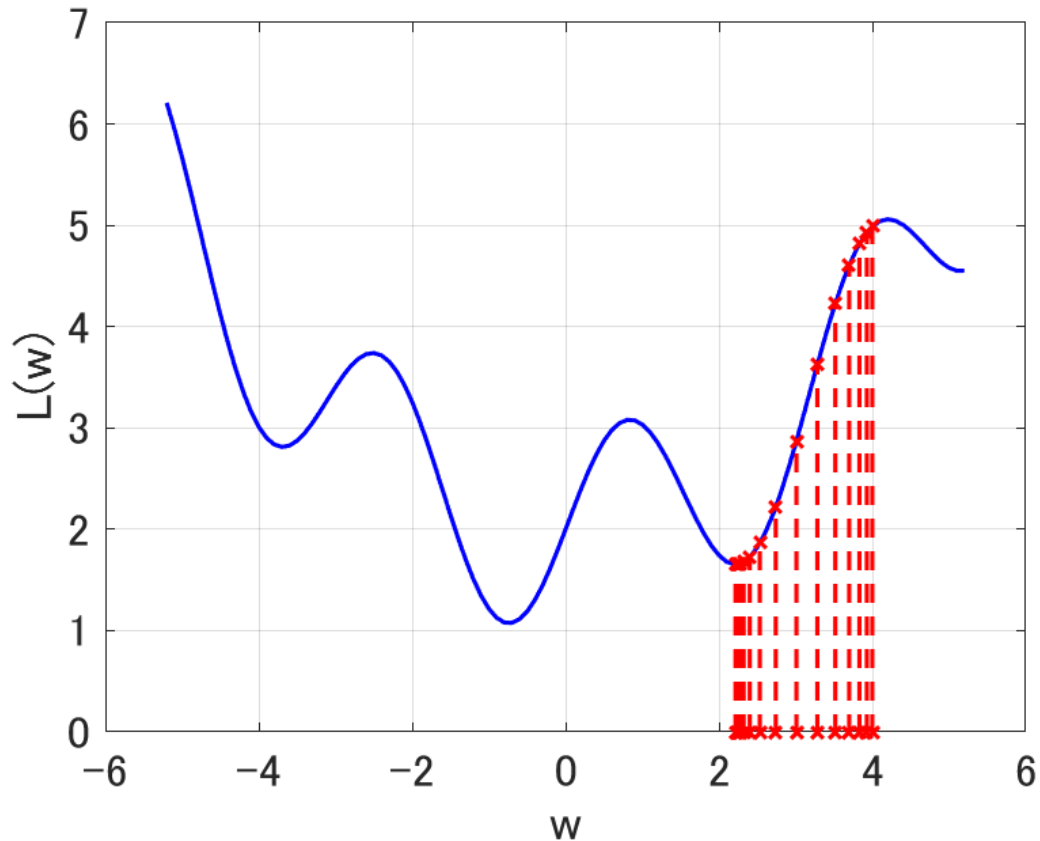


図 2.7: saikyu12

2.1.3 考察

(a) この演習課題をやるにあたり、 w 、 ep 、 $iters$ の値をそれぞれひとつだけ変化させ、出力されるグラフにどんな影響があるかを見た。 ep を変化させたとき、 ep が大きければ降下が早く、小さければ降下が遅いことが分かった (図 2.1、図 2.2)。次に w を変化させると、ただ初めに調べる場所が変化した (図 2.5、図 2.6) 期値よりも大きすぎると、極値を通り過ぎて反対に戻っている様子を伺うことができる。また、反復回数 $iters$ を変化させると探索する回数が変わることが分かる。(図 2.3、図 2.4)

(b) 目的関数を変更したためもちろん描画されるグラフも大きく異なっている。また、三角関数が含まれていることで波形がやや複雑になっている。描画したグラフを図 2.7 に示す。

2.2 演習課題 1.2

saikyu21.m のプログラムを以下にしめす。

Listing 2.2: saikyu21.m

```

1  % プログラム1.2: saikyu21.m
2  %
3  % 変数の目的関数対する最急降下法2
4  %
5  % すべての変数の値を初期化する。グラフをすべて閉じる。
6  clearvars, close all
7  % ← (A)
8  %% 最急降下法
9  % 初期値, ステップ幅, 反復回数の設定
10 w = [1,1]; % の初期値w
11 ep = 0.1; % ステップ幅
12 iters = 20; % 反復回数 (number of ) iterations
13 ws = w; % ws: w のすべての値を保存する
14 % 更新の反復
15 for i=1:iters
16   gk = grad(w(1),w(2)); % 勾配の計算
17   w = w - ep*gk; % 更新
18   ws = [ws; w]; % 反復点の保存
19 end
20
21 %% 目的関数のグラフを描く
22 % w: 変数. 範囲: から刻みでまでのメッシュ範囲 -60.256
23 % LW: 目的関数
24 [w1, w2] = meshgrid(-6:0.25:5.5);
25 LW = obj_func(w1, w2);
26 % 目的関数の描画
27 sc = meshc(w1, w2, LW); hold on
28 sc(2).LevelList = 2:2:40; % (w1, w2) 平面での等高線の間隔を指定
29 xlabel('w1'), ylabel('w2'), zlabel('L(W)')
30 set(gca, 'FontSize', 14)
31 % ← (B)
32 %% 結果の描画
33 for i=1:iters
34   yk = obj_func(ws(i,1), ws(i,2));
35   plot3([ws(i,1), ws(i,1)], [ws(i,2), ws(i,2)], [0, yk], 'rx--', 'LineWidth', 1.5)
36 end
37 %% (w1, w2) 平面での等高線の描画
38 figure % 新しい描画ウィンドウを開く
39 contour(w1, w2, LW, 25), hold on
40 plot(ws(:,1), ws(:,2), 'rx-', 'LineWidth', 1.5)
41 xlabel('w1'), ylabel('w2')
42 set(gca, 'FontSize', 14)
43 %% 目的関数の計算
44 function y = obj_func(w1, w2)
45   y = (w1.^4)/24 + (w2.^4)/32 - w1.^2 - w2.^2 + w1 + w2 + 25;
46 end
47 %
48 %% 勾配の計算
49 function g = grad(w1, w2)
50   g = [(w1.^3)/6 - 2*w1 + 1, (w2.^3)/8 - 2*w2 + 1];
51 end
52 %

```

(a) 初期値 w やステップ幅 ep を様々な値に変更し、最急降下法での収束がどのように変わるかを確かめよ。必要に応じて、反復回数 $iters$ も変更する。

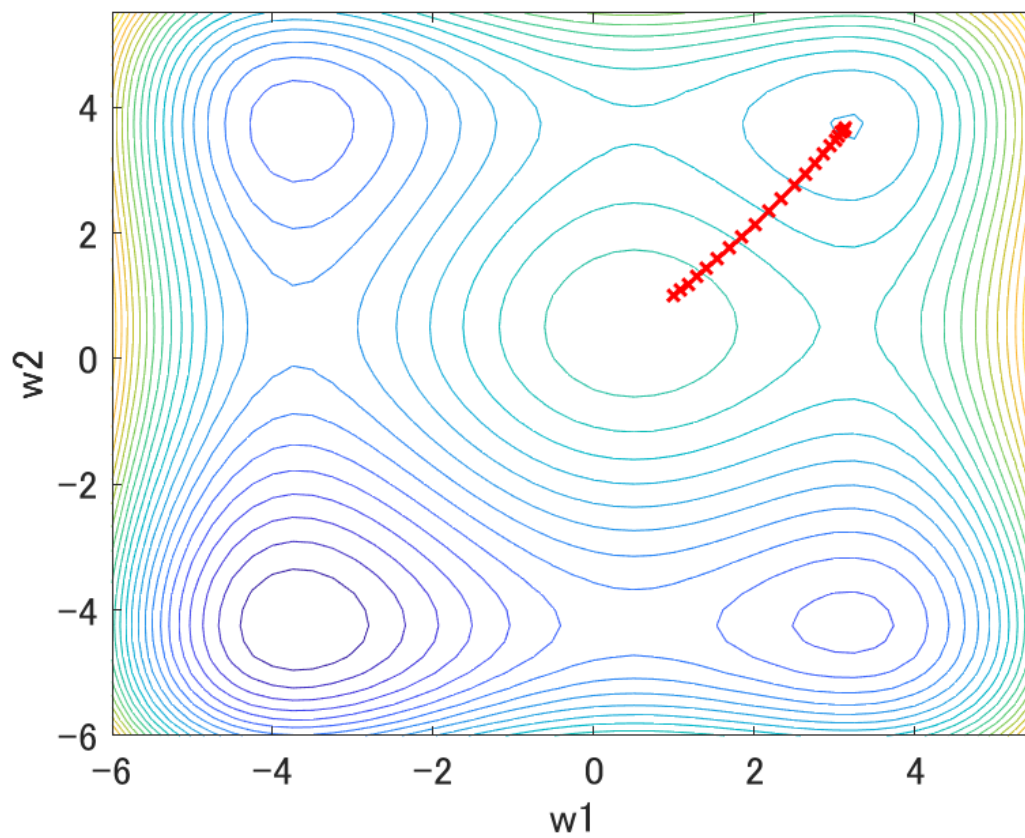


図 2.8: saikyu21

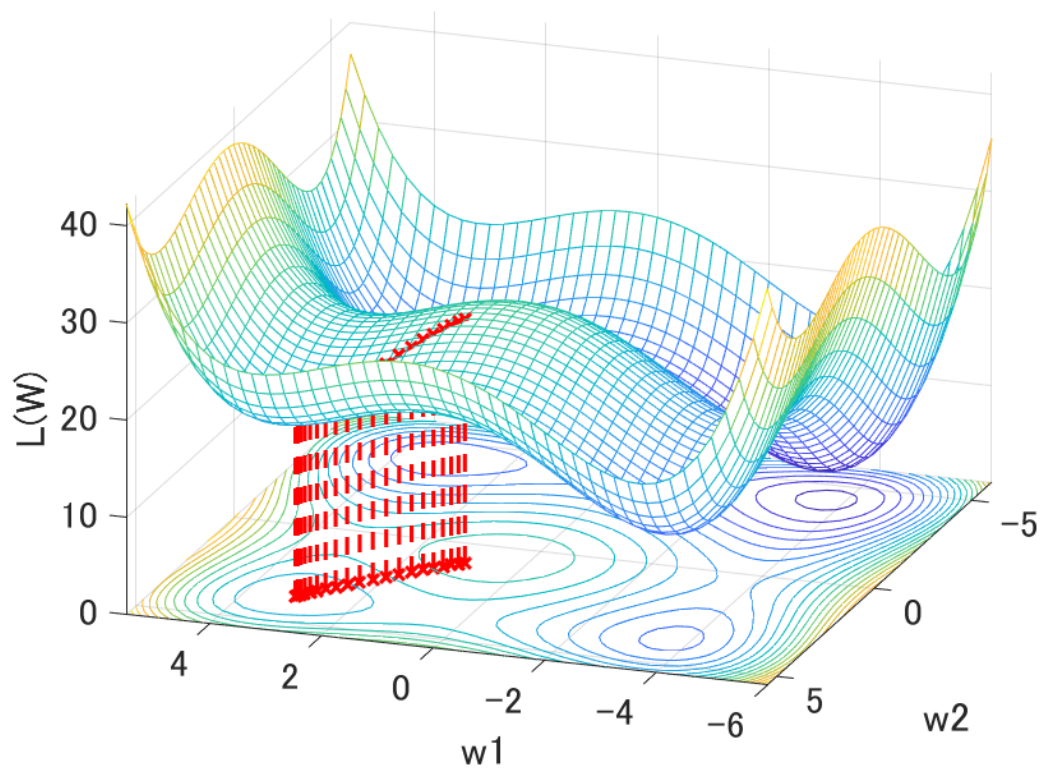


図 2.9: saikyu21

2.2.1 考察

完成したプログラムを実行すると画像（図 2.8,2.9）のように出力が得られた。初期値 W 、ステップ幅 ep を様々な値に変更して最急降下法での収束がどのように変わるかを調べたところ、演習課題 1.1 で述べた考察と同じ変化が見られた。

3 第2週：パーセプトロン

3.1 演習課題 2.1

3.1.1 (a)

(a) 初期値 W, b を変化させて収束の様子を確認する。必要に応じてエポック数を変更する。また、誤差の範囲を変更し、完全にはクラス分類できない場合があることを確かめよ。

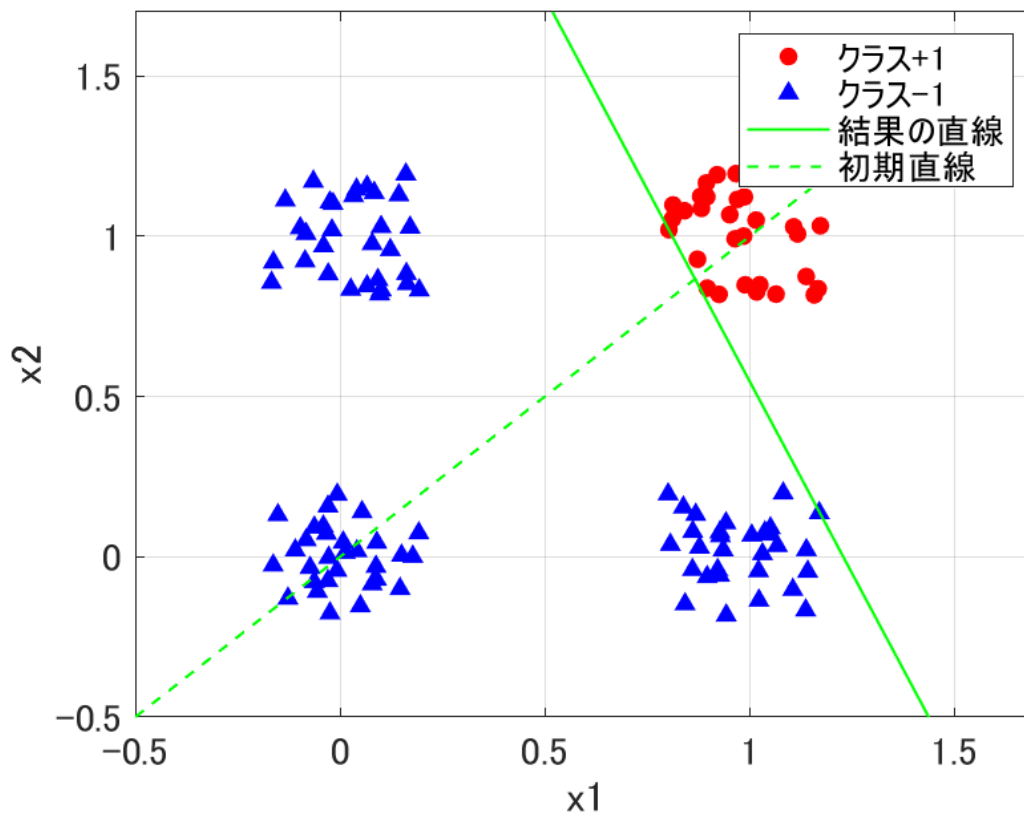


図 3.1: $er=0.2$

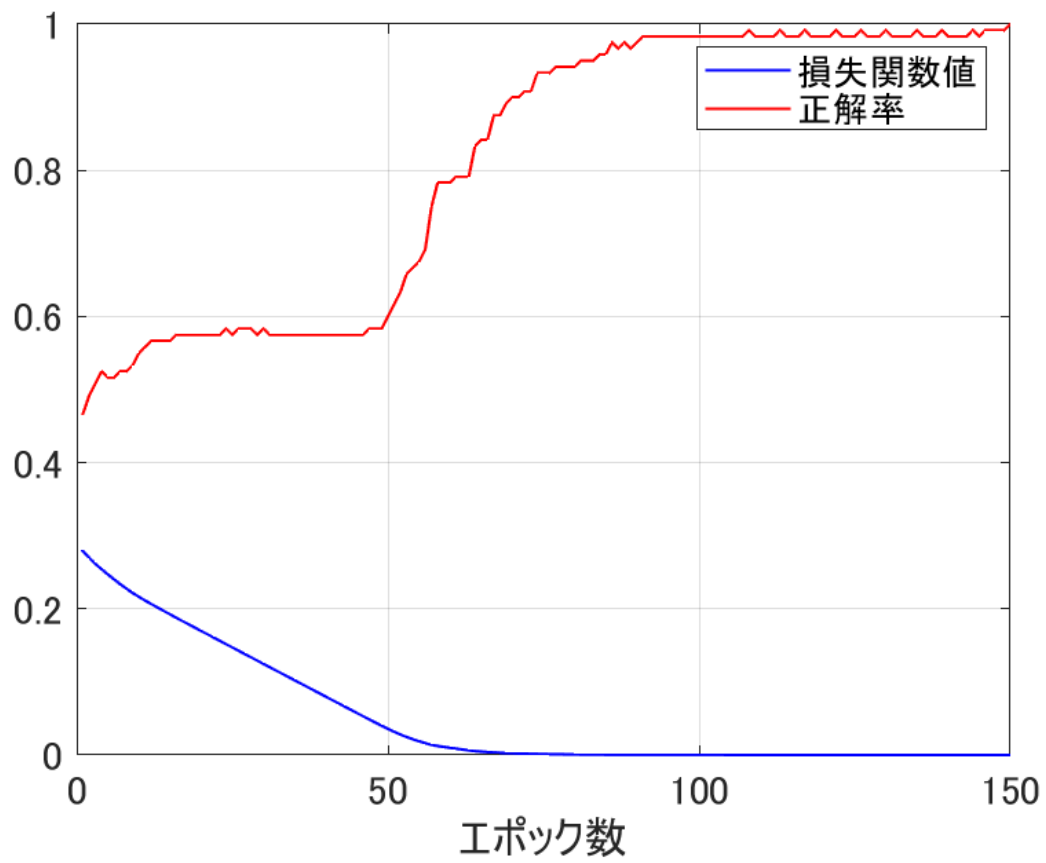


図 3.2: $er=0.2$

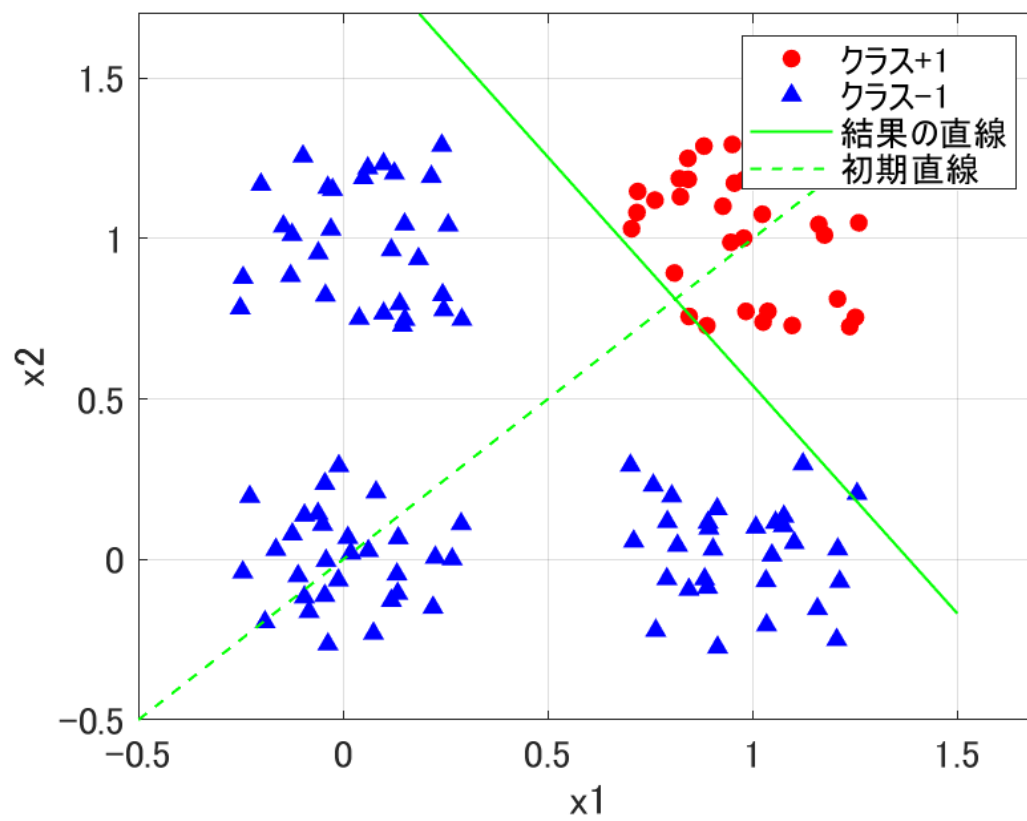


図 3.3: $er=0.3$

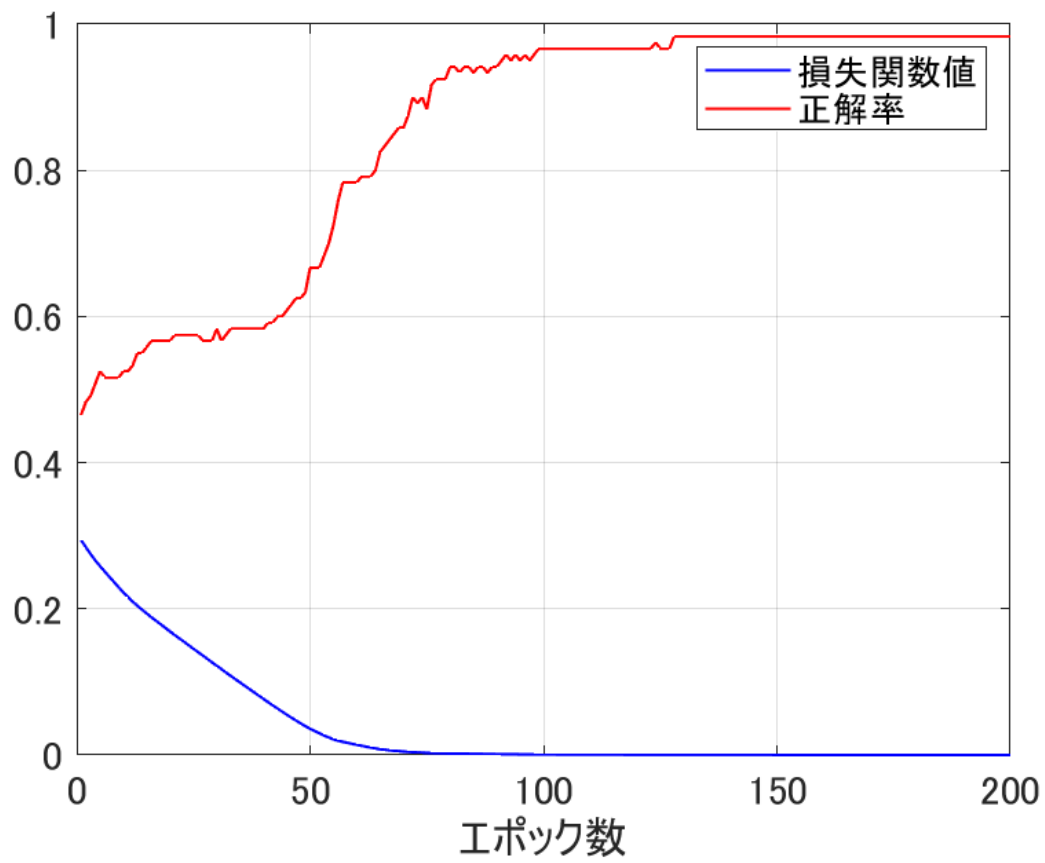


図 3.4: $er=0.3$

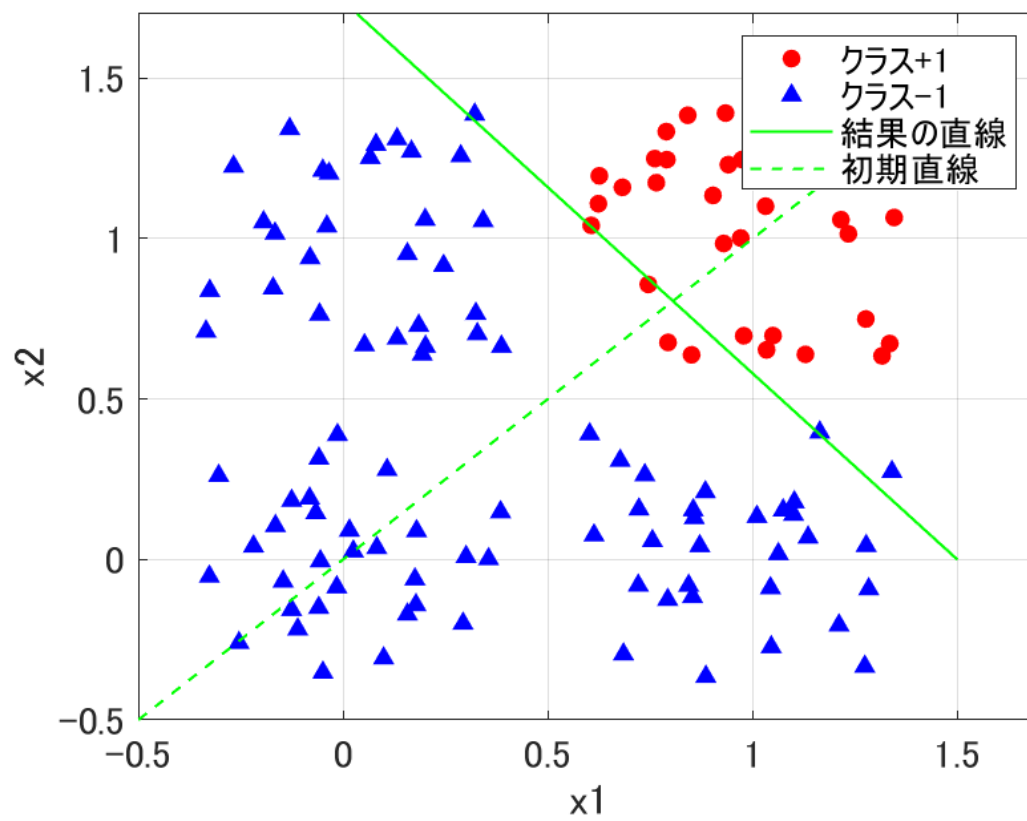


図 3.5: $er=0.4$

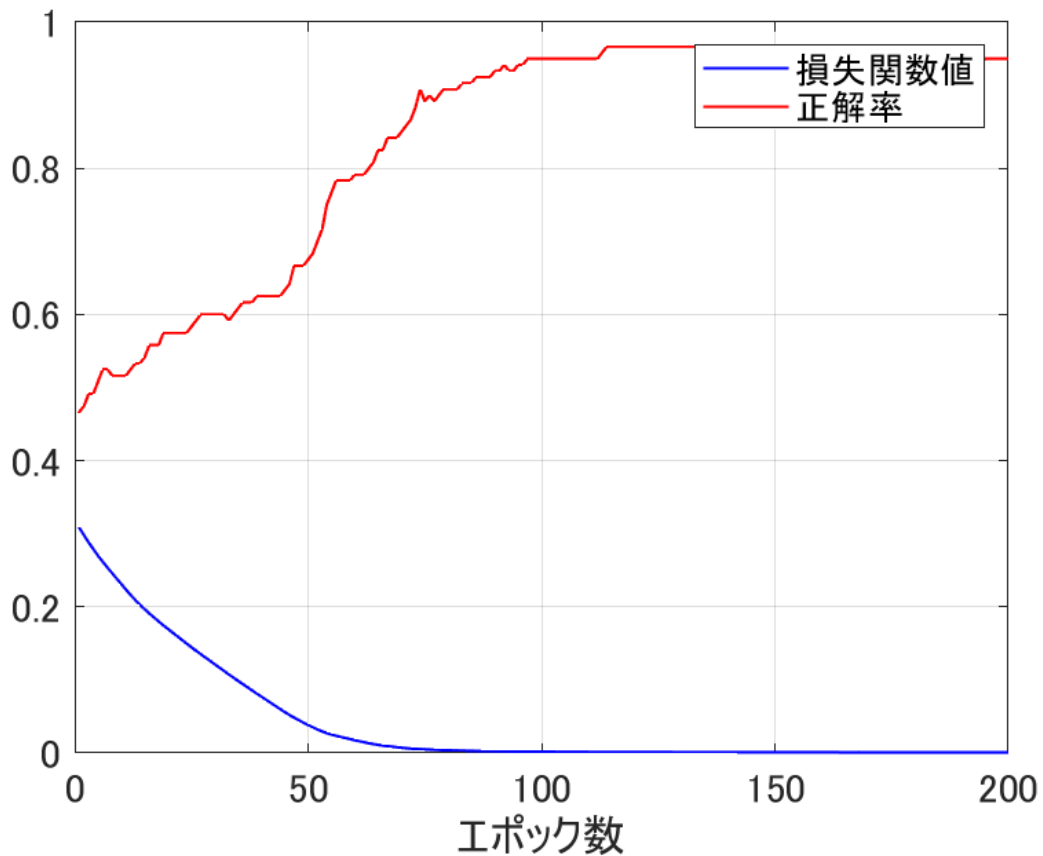


図 3.6: $er=0.4$

3.1.2 考察 (a)

直線の傾きを W が、切片を b が表している。また er は点のばらつきの度合いを表している。 er の値が大きいほどばらつきが大きくなる。私は今回この考察を書くにあたり er が 0.2, 0.3, 0.4 のときについて調べた。添付している画像の損失関数を見ると、40 から 200 ほどまでには 0 になっていて、期待された値とのずれがないことが見て取れる。このことからこれ以上学習させても正解率が上がることが期待されないため、これ以上のクラスタリングは不可能であると考えられる。

3.1.3 (b)

(b) 入力の中ほどにある $tset$ を or 関係にして、(a) と同様に収束性を確認する。

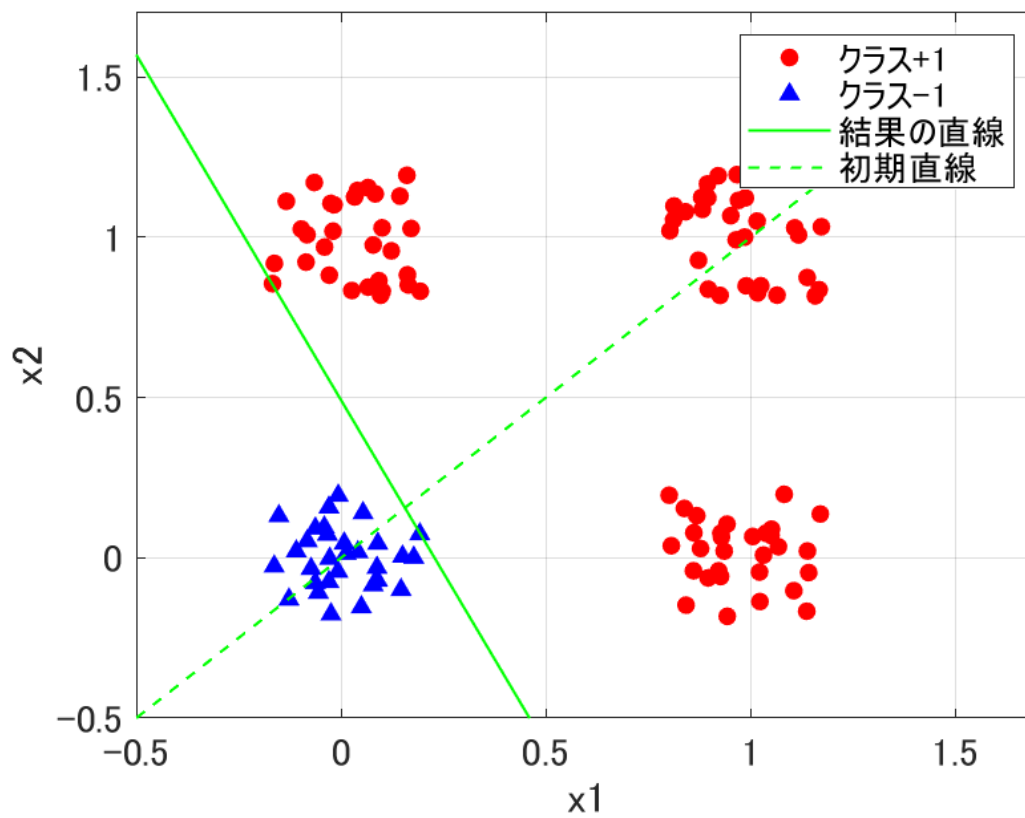


図 3.7: or 関係

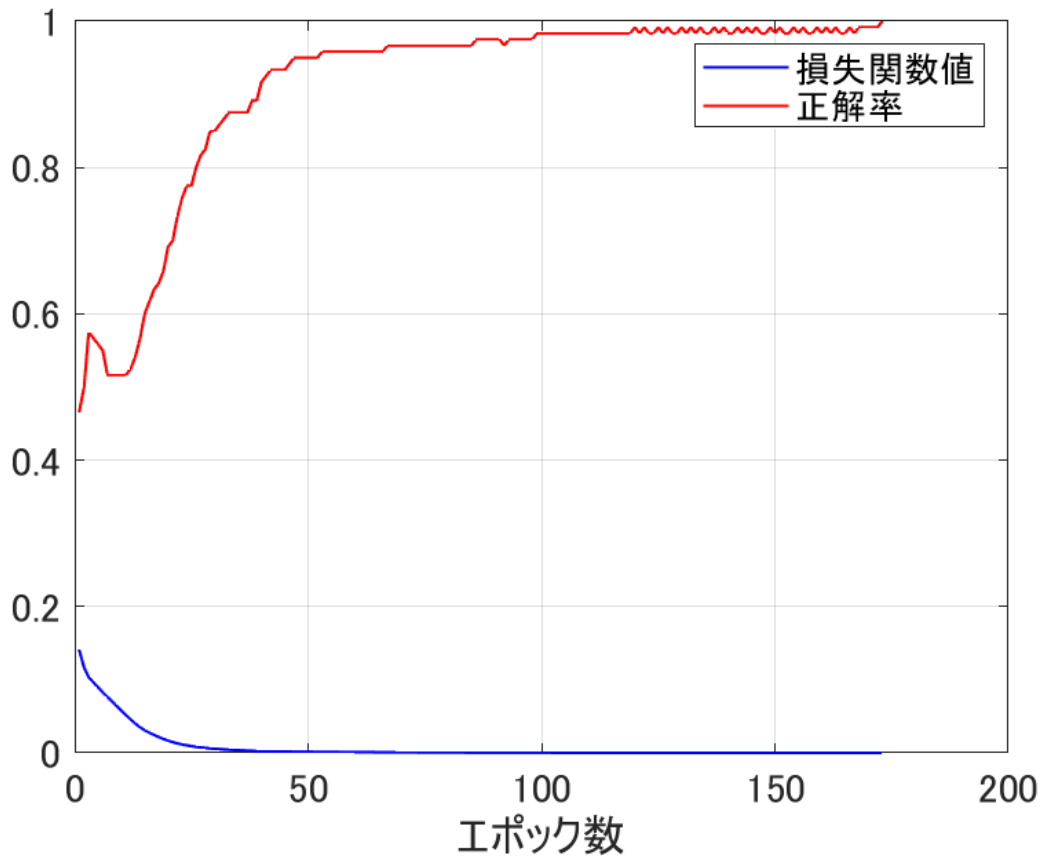


図 3.8: or 関係

3.1.4 考察 (b)

and 関係から or 関係に変更して、プログラムを実行した。実行して得られた損失関数と正解率のグラフを見ると、収束の仕方は and 関係のグラフと変わらないように見える。

3.1.5 (c)

(c) 上記プログラムをオンライン学習やミニバッチ学習に変更せよ。ミニバッチ学習の場合、例えばバッチサイズを 20 や 30 として実行せよ。

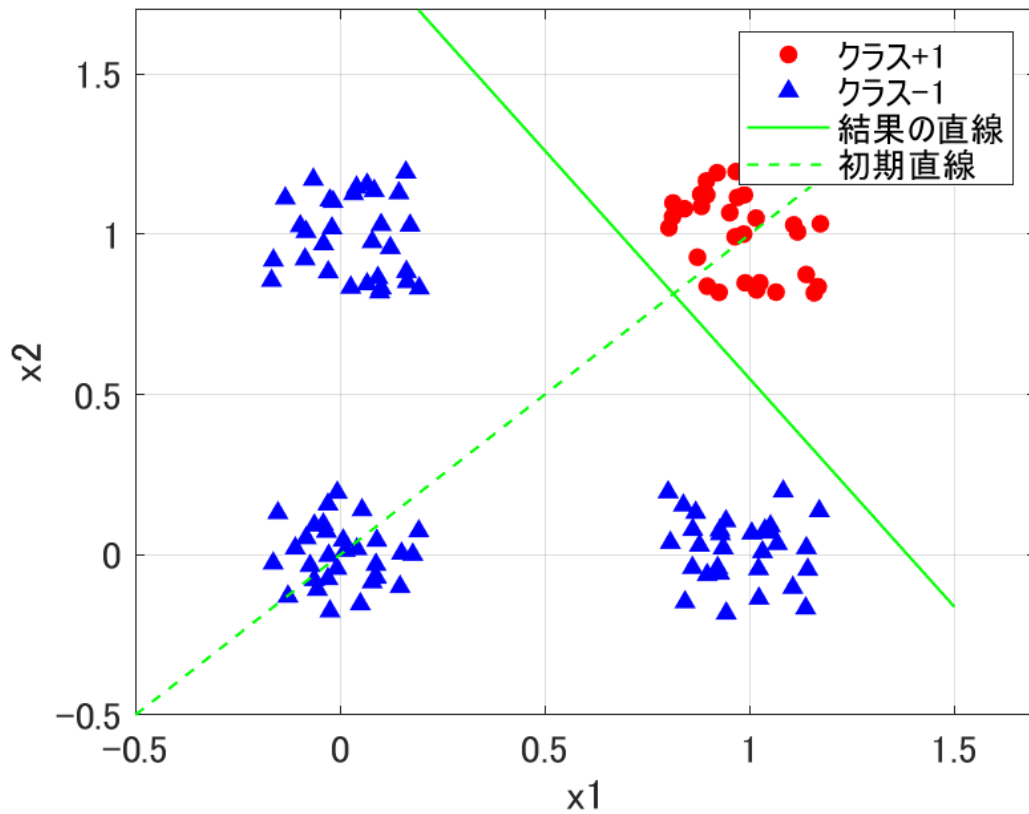


図 3.9: オンライン学習

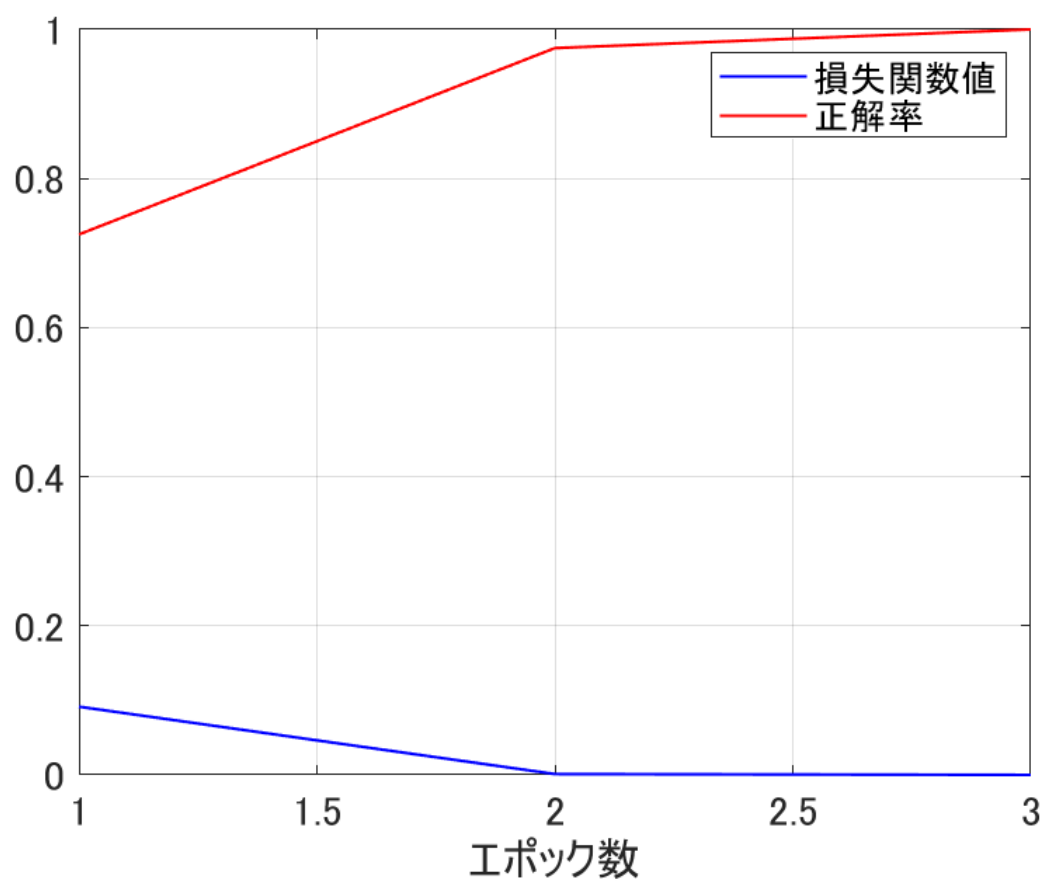


図 3.10: オンライン学習

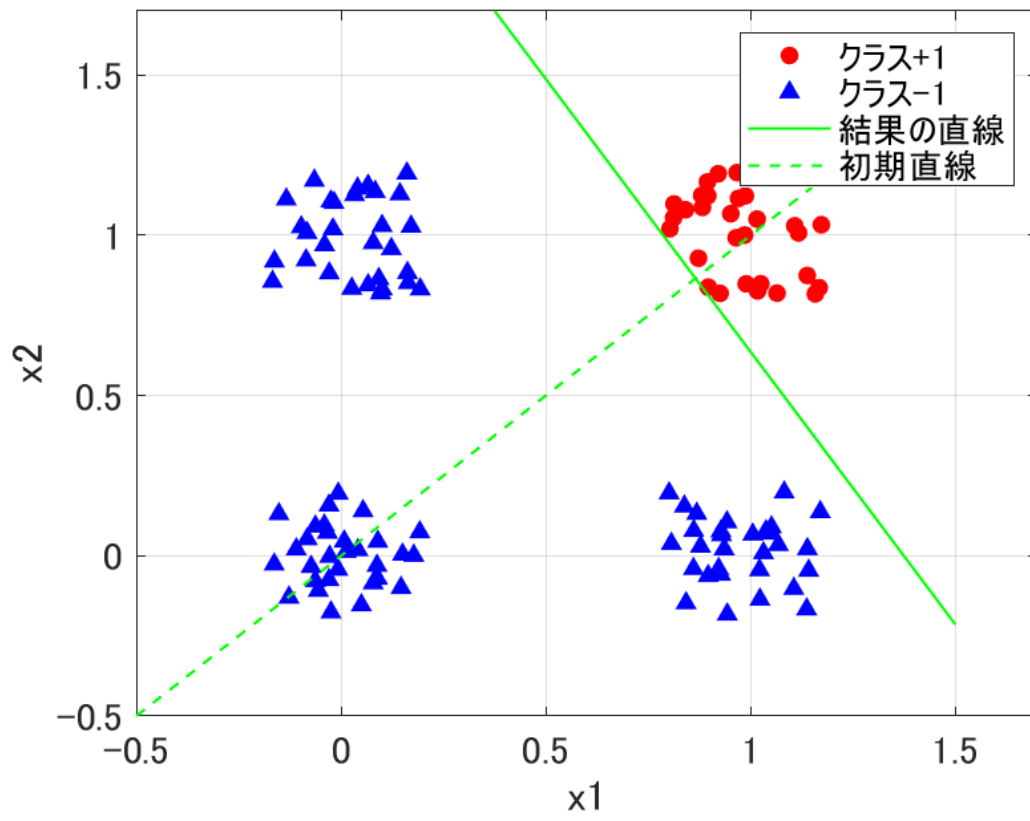


図 3.11: ミニバッチ学習

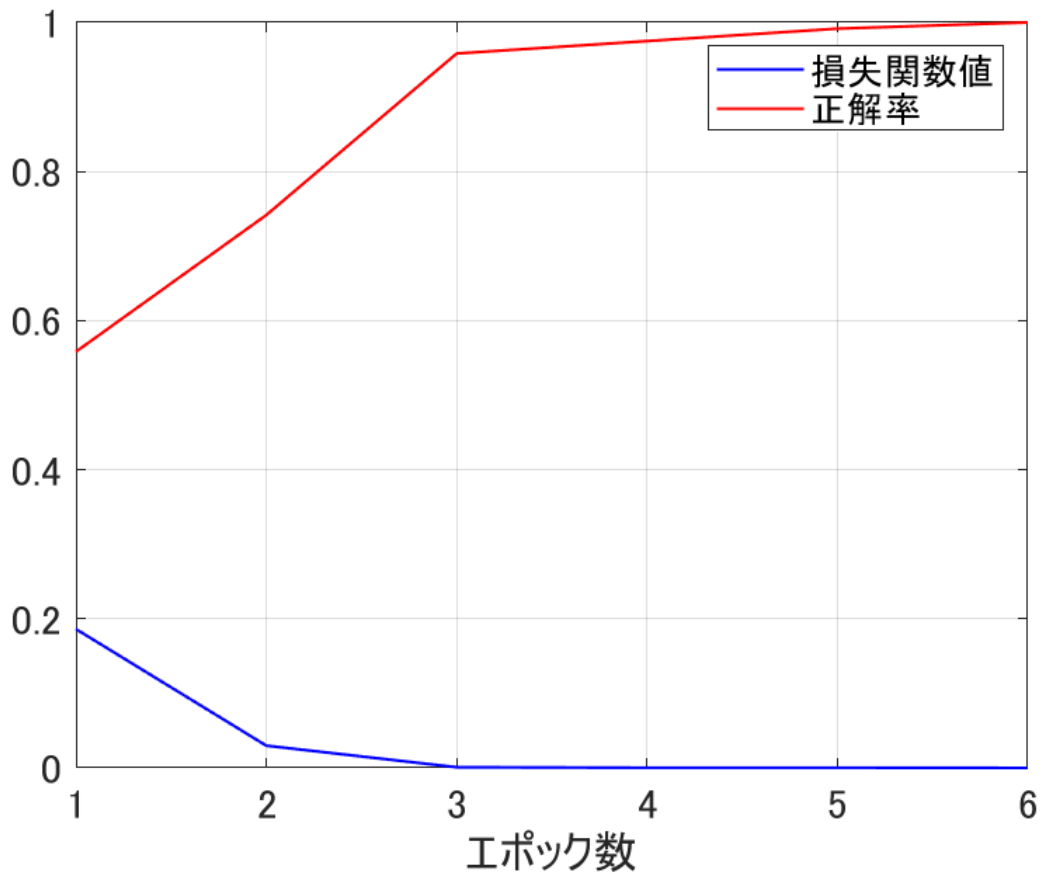


図 3.12: ミニバッチ学習

3.1.6 考察 (c)

オンライン学習とミニバッチ学習はどちらもバッチ学習よりもエポック数が少なくなる学習方法ですが、ある一定の区間をまとめてバッチ学習をして、その処理を繰り返しているミニバッチ学習のほうが、オンライン学習よりもエポック数が多くなっている。それぞれのアルゴリズムをみればこの結果が出た理由が良くわかると思った。特にバッチ学習は全データを見ることでエポック数が一番多くなる、という結果が顕著にグラフに表れているよ

うに見える。

4 第3週：多層ニューラルネットワーク

4.1 演習課題 3.1 (a)

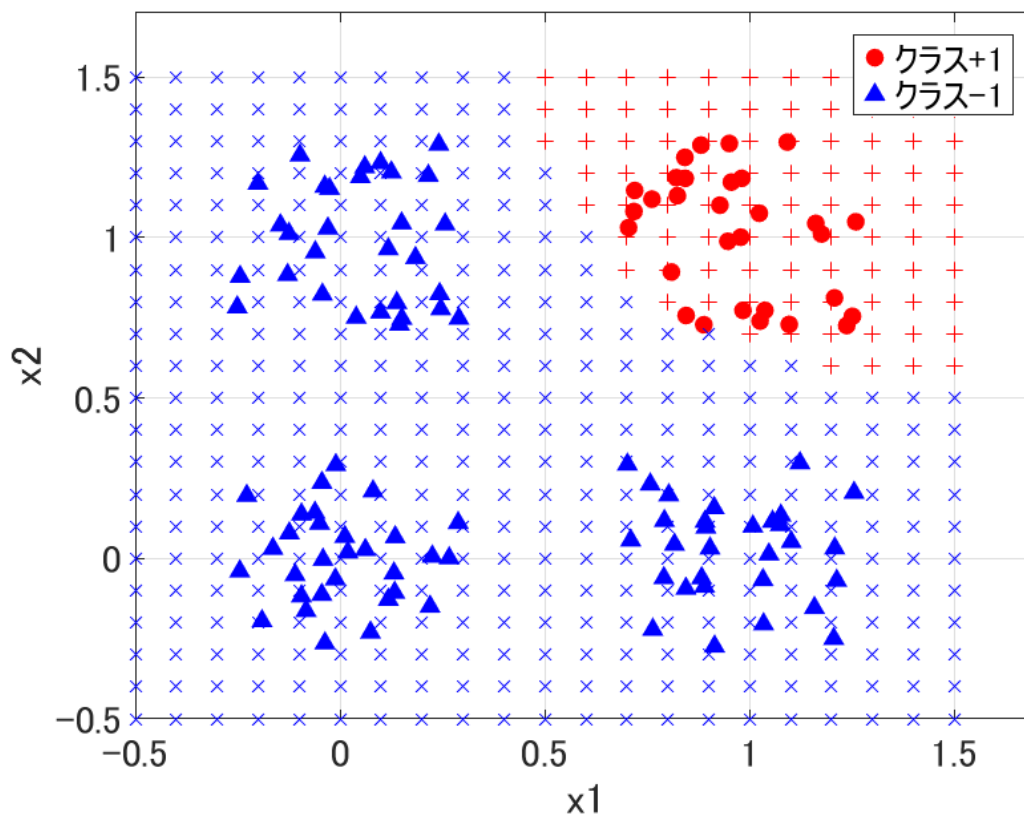


図 4.1: 3 層 NN, dimlayer=50, numEpochs=200, er=0.3

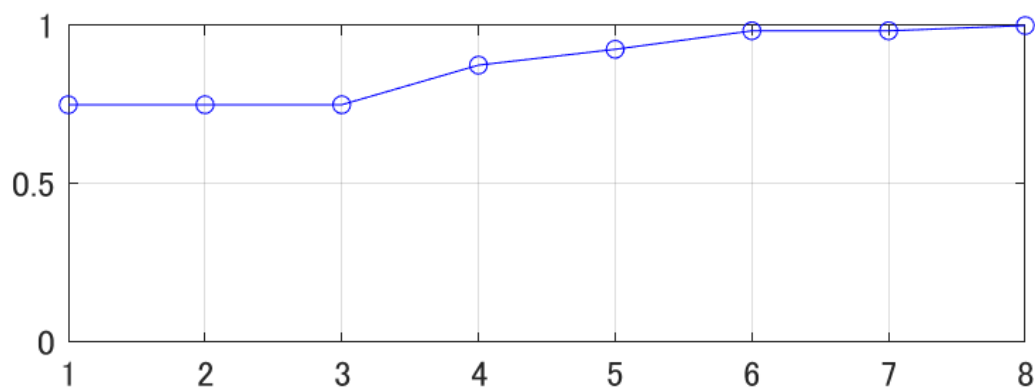


図 4.2: 3 層 NN, dimlayer=50, numEpochs=200, er=0.3

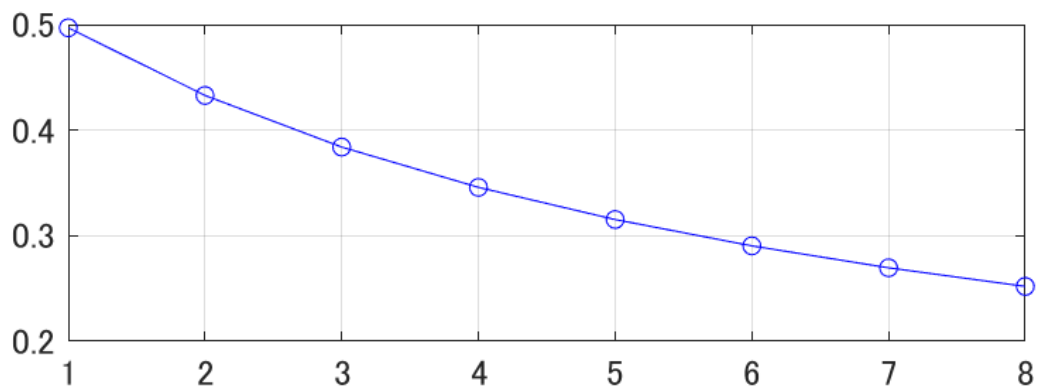


図 4.3: 3 層 NN, dimlayer=50, numEpochs=200, er=0.3

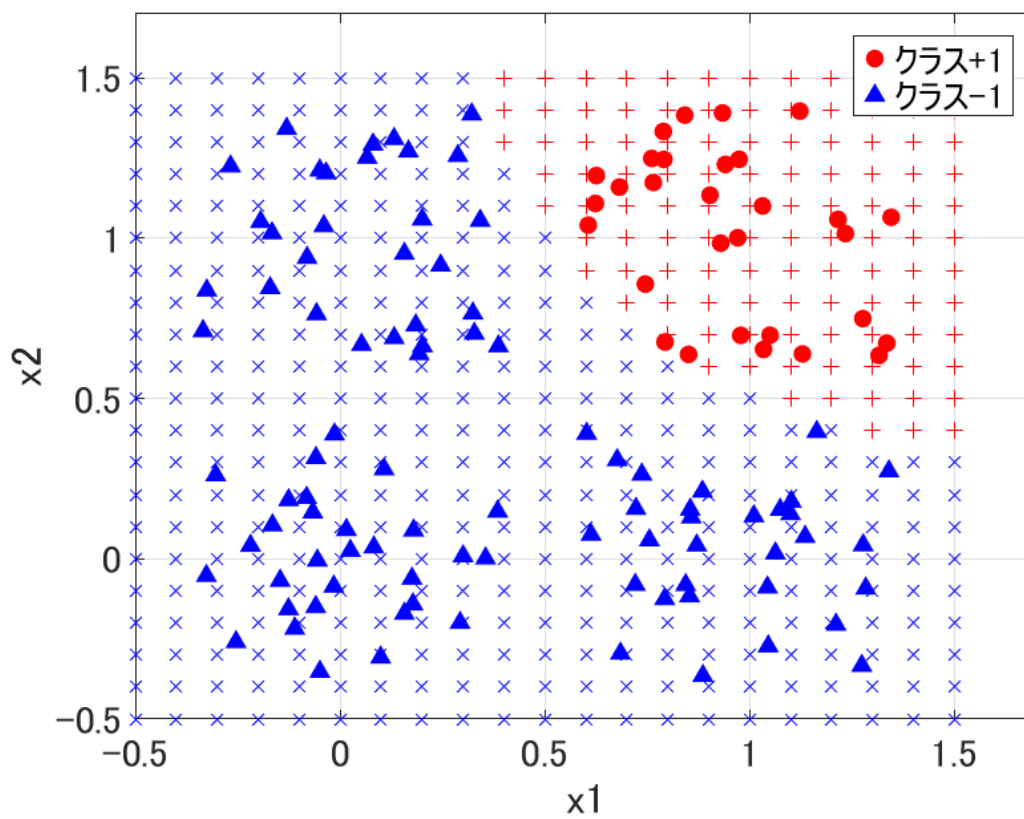


図 4.4: 3 層 NN, dimlayer=50, numEpochs=200, er=0.4

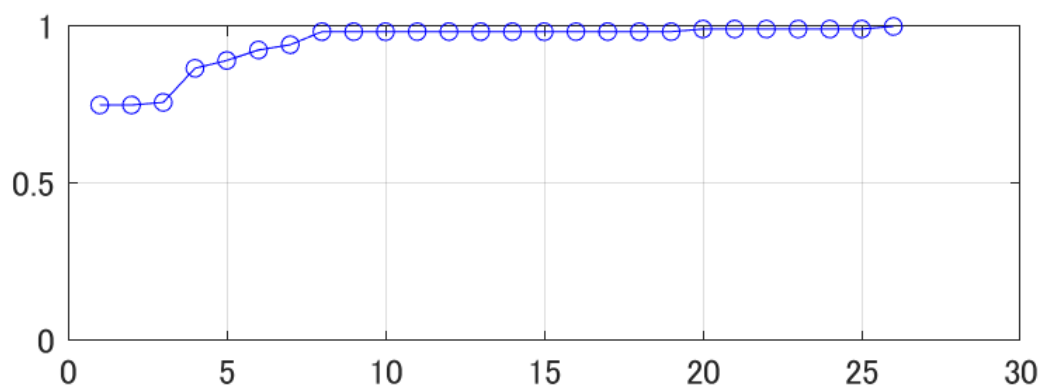


图 4.5: 3 層 NN, dimlayer=50, numEpochs=200, er=0.4

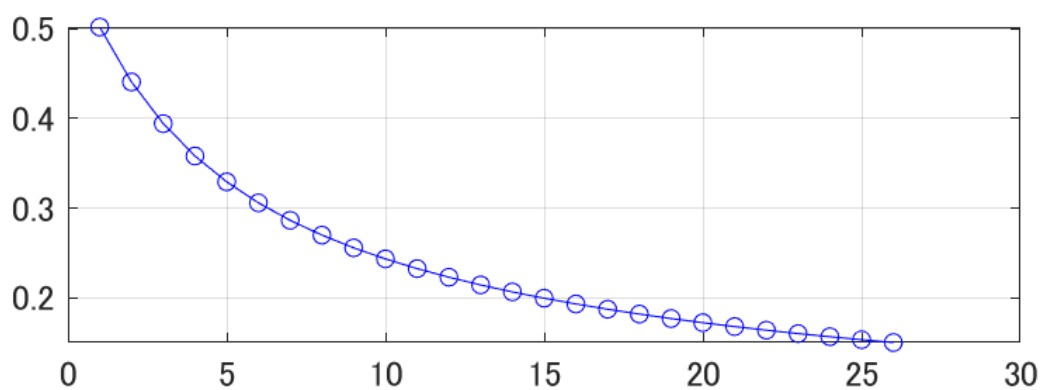


图 4.6: 3 層 NN, dimlayer=50, numEpochs=200, er=0.4

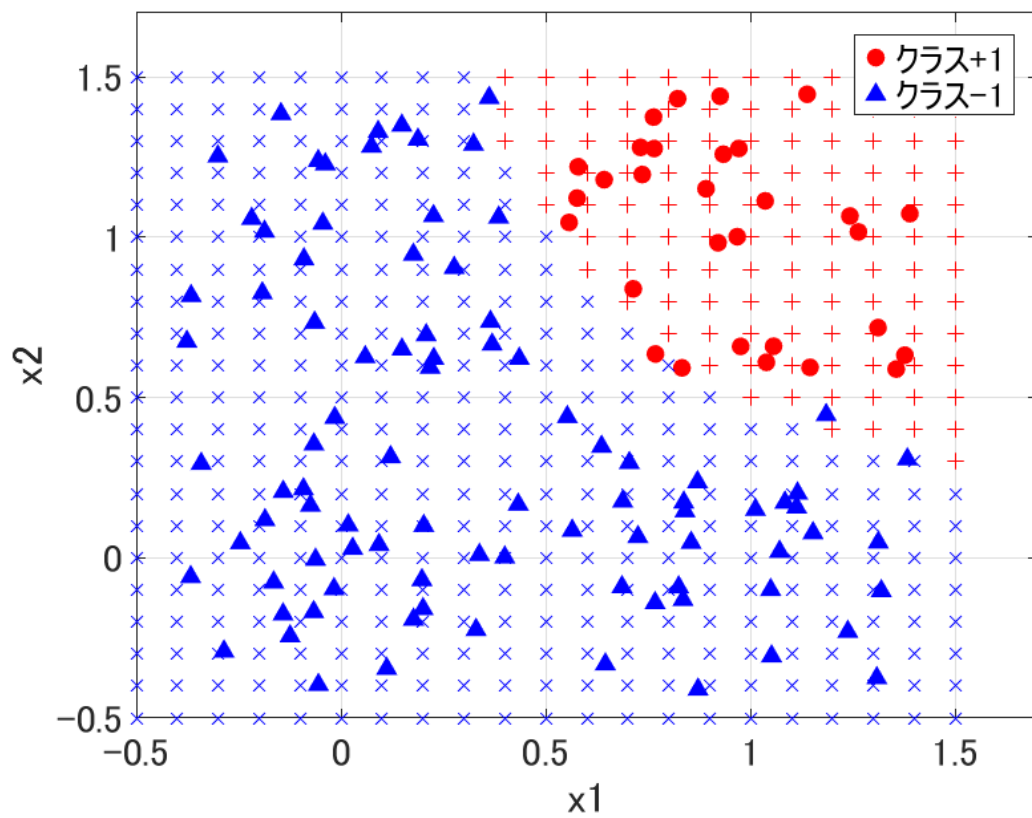


図 4.7: 3 層 NN, dimlayer=50, numEpochs=200, er=0.45

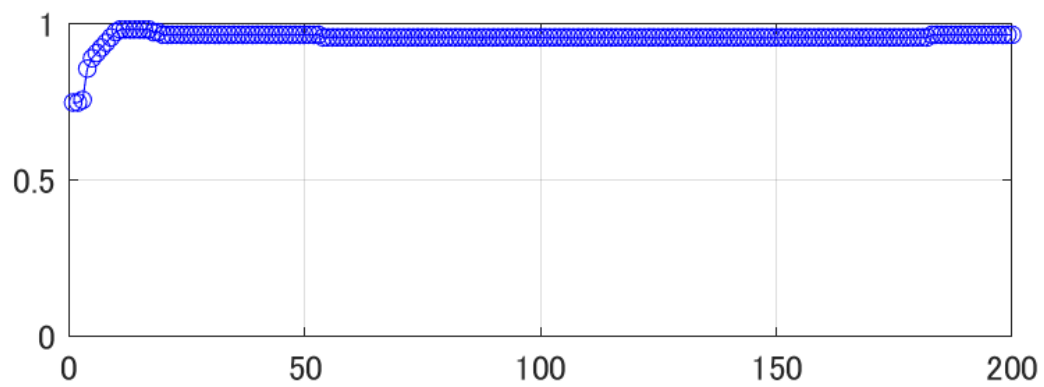


図 4.8: 3 層 NN, dimlayer=50, numEpochs=200, er=0.45

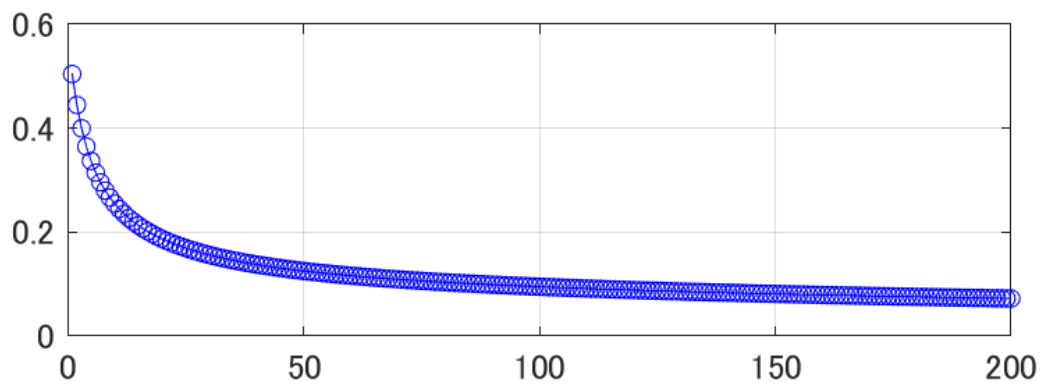


図 4.9: 3 層 NN, dimlayer=50, numEpochs=200, er=0.45

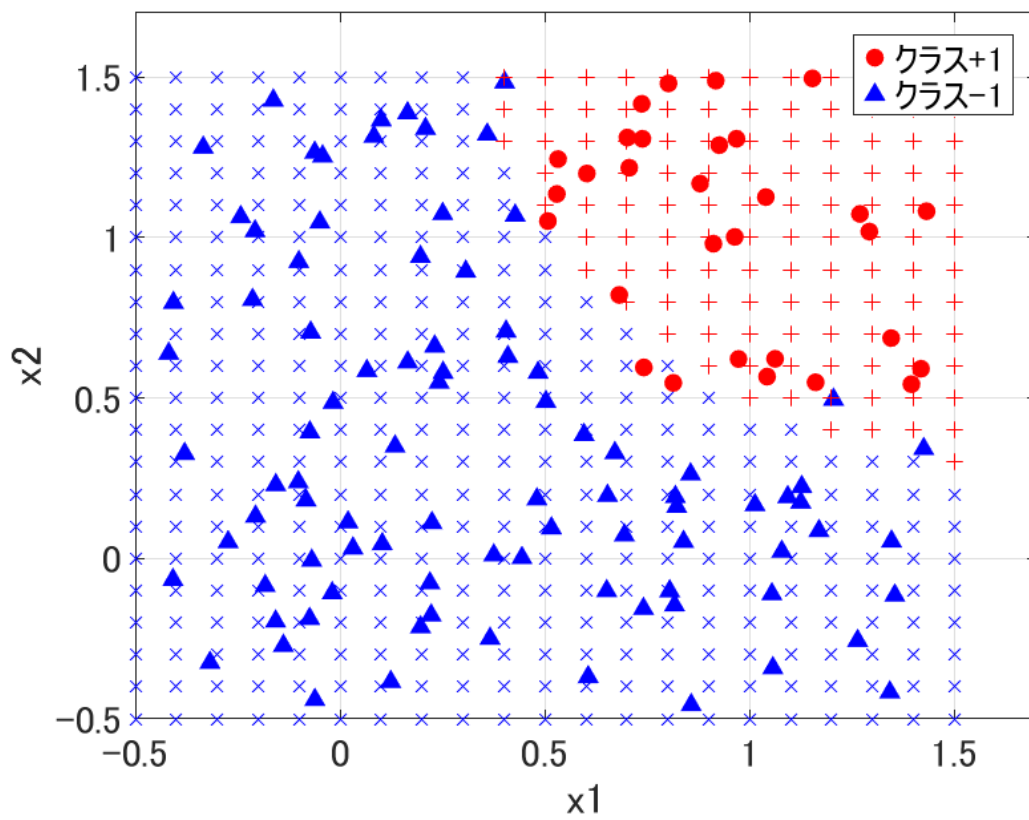


図 4.10: 3 層 NN, dimlayer=50, numEpochs=200, er=0.5

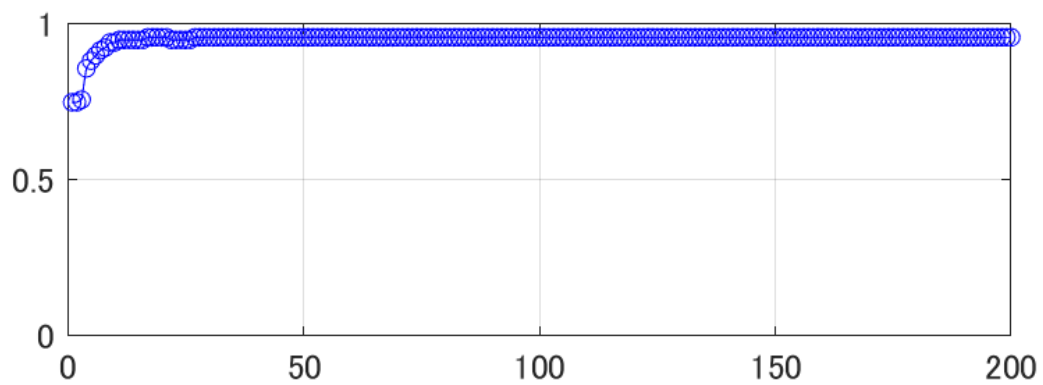


図 4.11: 3 層 NN, dimlayer=50, numEpochs=200, er=0.5

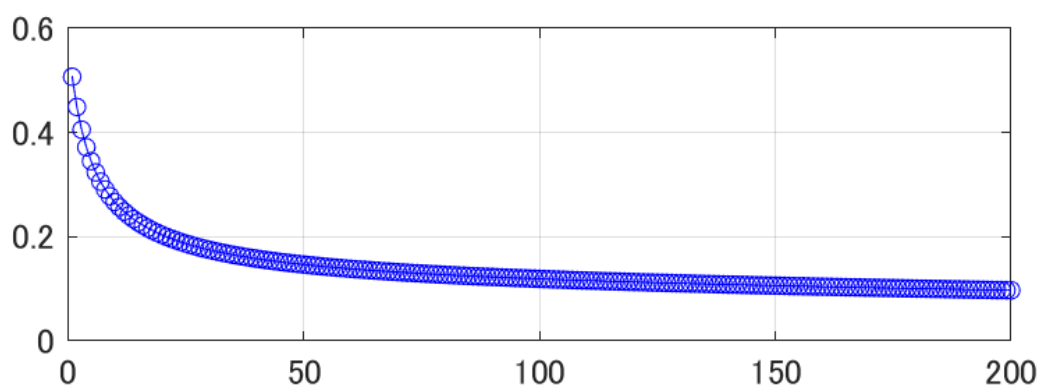


図 4.12: 3 層 NN, dimlayer=50, numEpochs=200, er=0.5

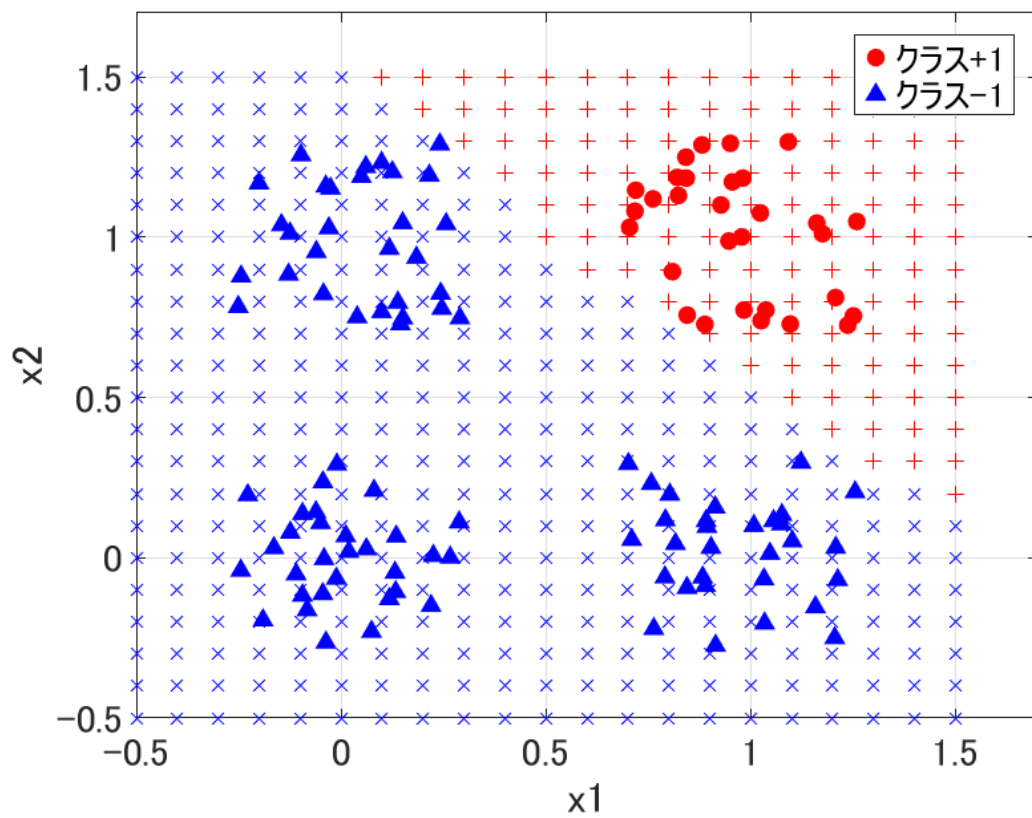


図 4.13: 3 層 NN, dimlayer=100, numEpochs=200, er=0.3

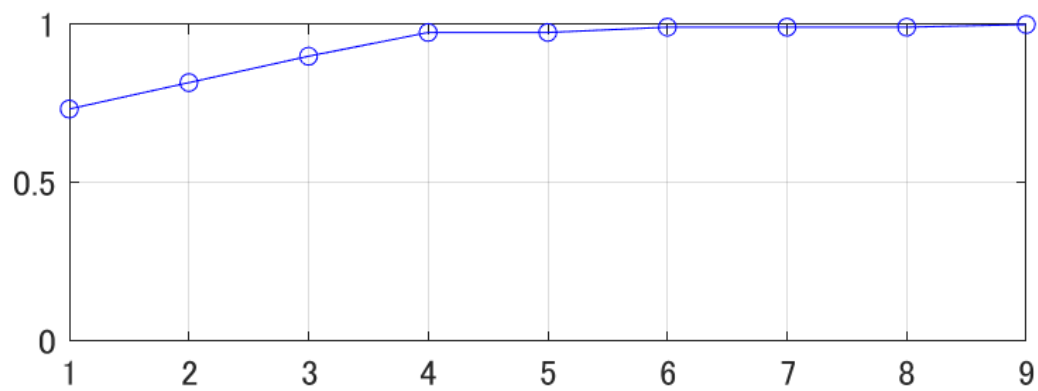


図 4.14: 3 層 NN, dimlayer=100, numEpochs=200, er=0.3

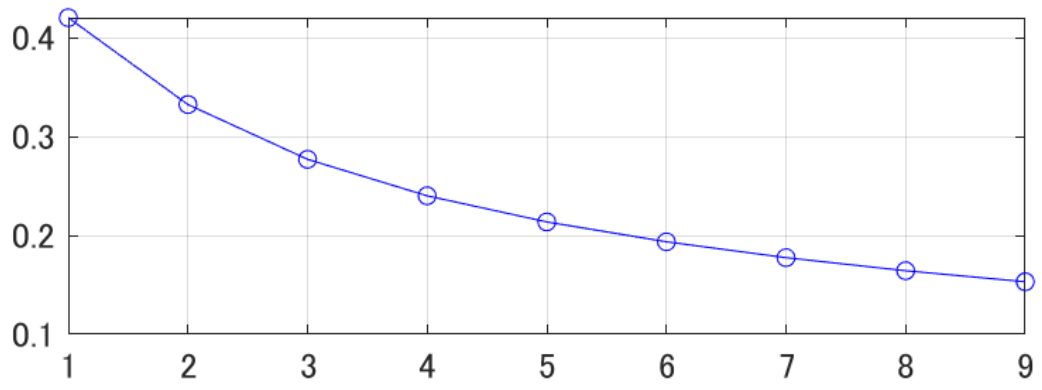


図 4.15: 3 層 NN, dimlayer=100, numEpochs=200, er=0.3

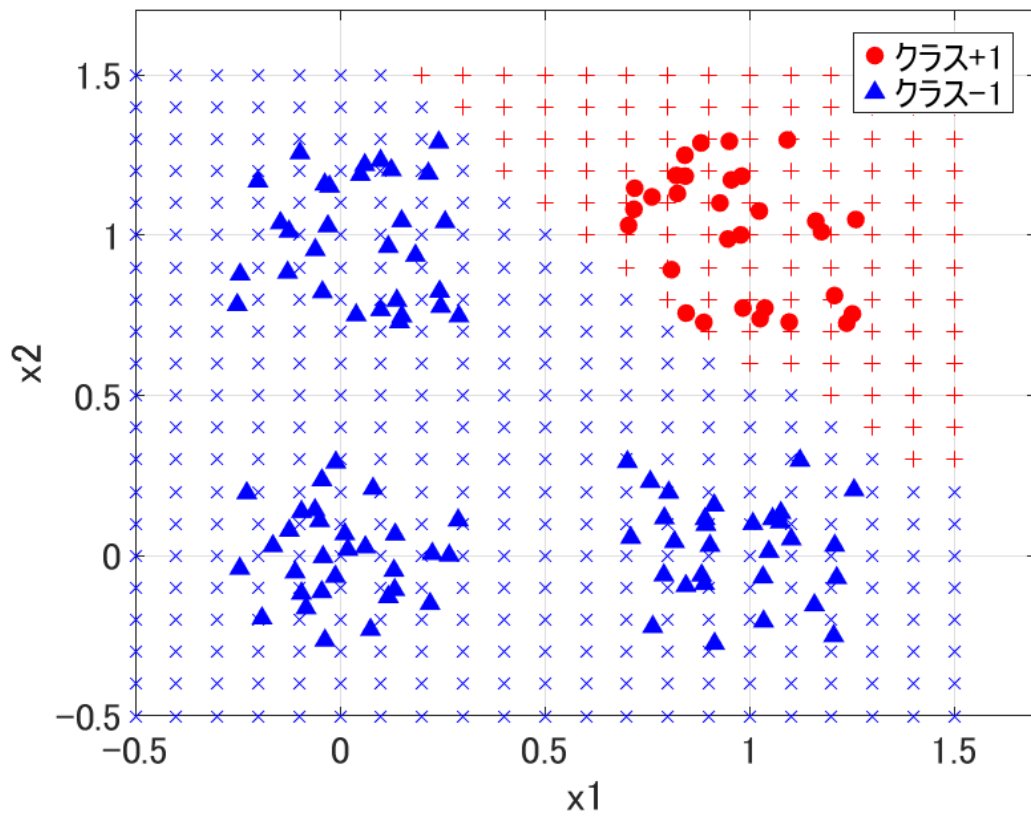


図 4.16: 3 層 NN, dimlayer=200, numEpochs=200, er=0.3

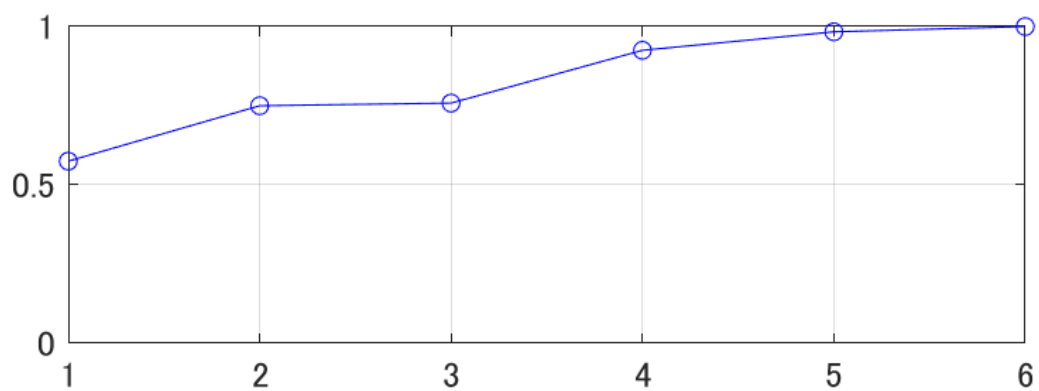


図 4.17: 3 層 NN, dimlayer=200, numEpochs=200, er=0.3

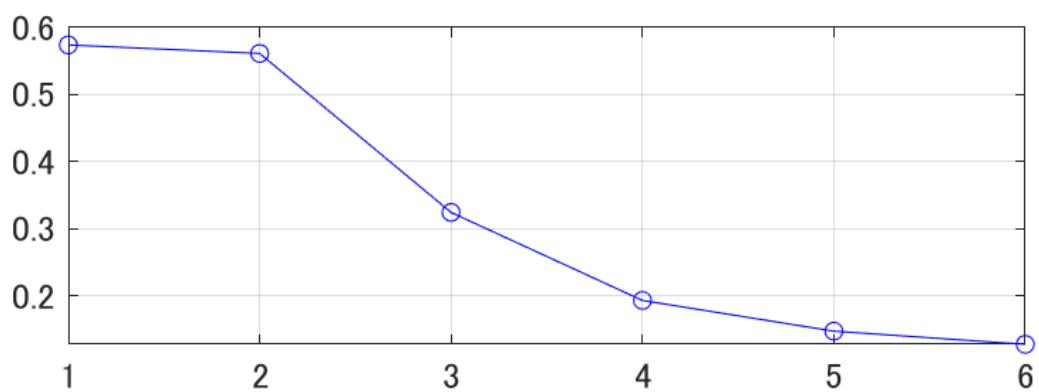


図 4.18: 3 層 NN, dimlayer=200, numEpochs=200, er=0.3

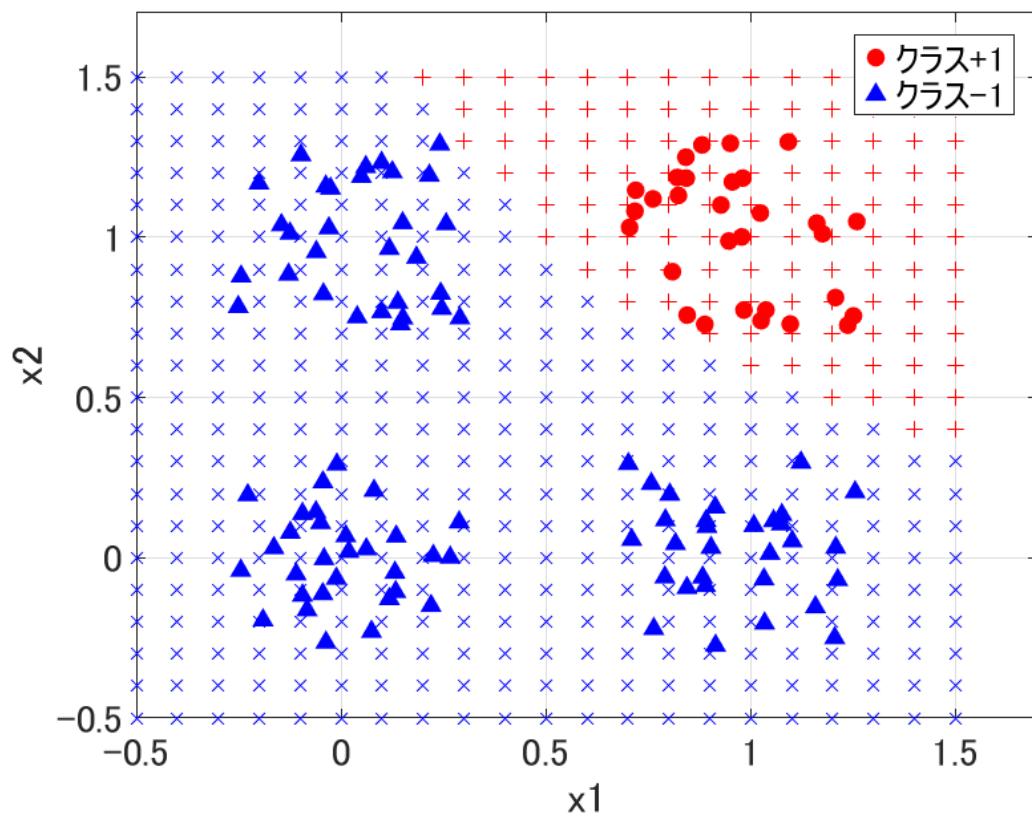


図 4.19: 3 層 NN, dimlayer=500, numEpochs=200, er=0.3

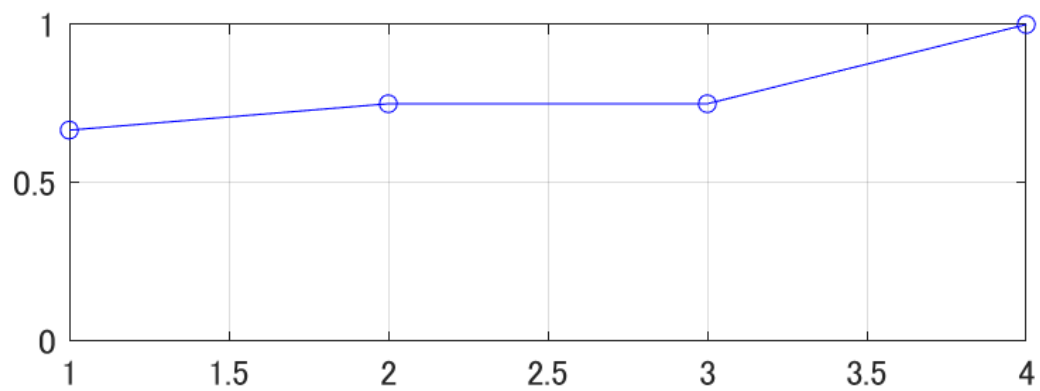


図 4.20: 3 層 NN, dimlayer=500, numEpochs=200, er=0.3

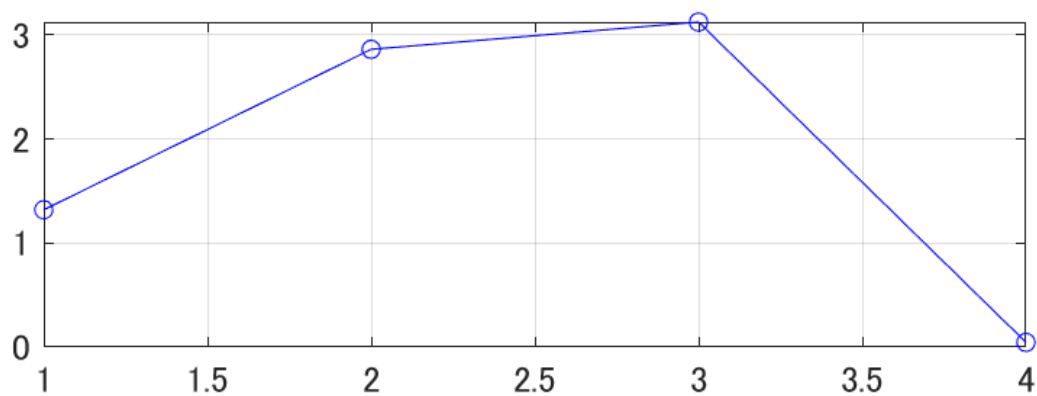


図 4.21: 3 層 NN, dimlayer=500, numEpochs=200, er=0.3

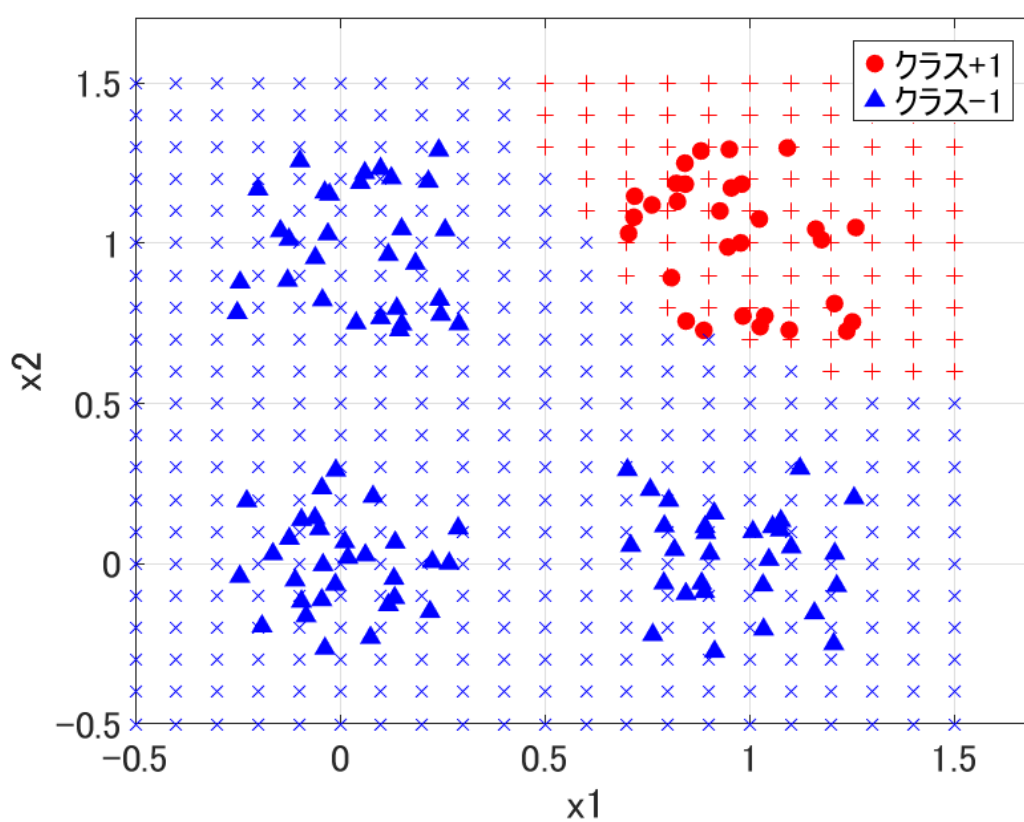


図 4.22: 3 層 NN, dimlayer=500, numEpochs=100, er=0.3

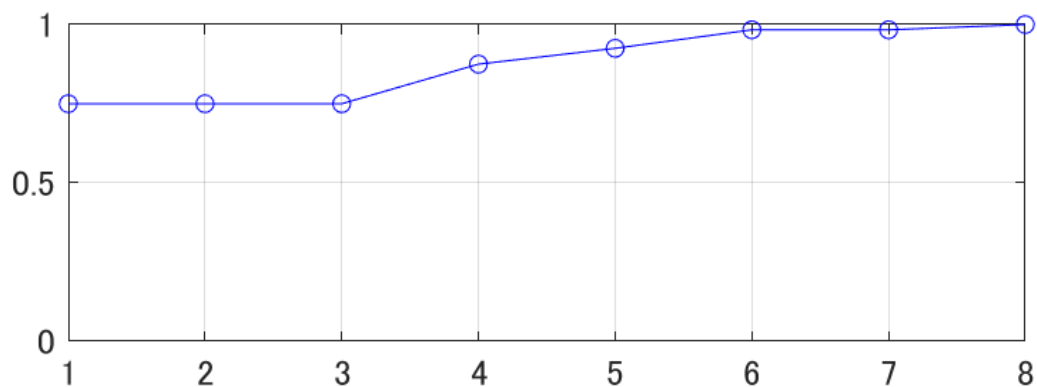


図 4.23: 3 層 NN, dimlayer=500, numEpochs=100, er=0.3

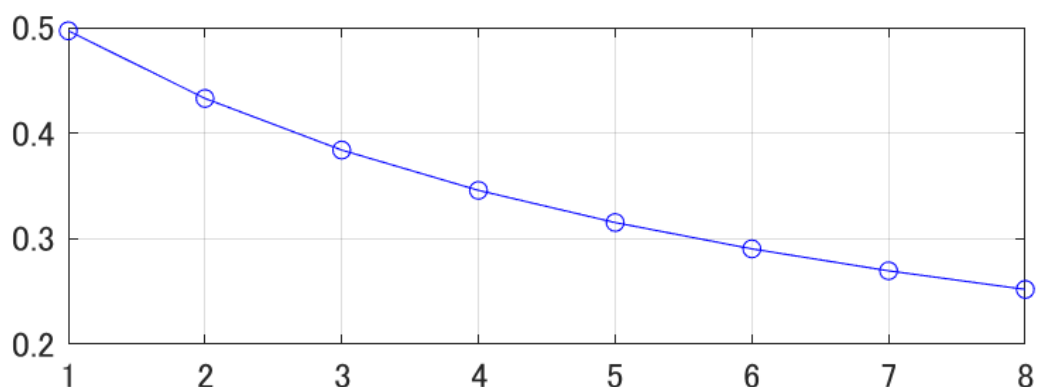


図 4.24: 3 層 NN, dimlayer=500, numEpochs=100, er=0.3

4.2 考察 (a)

誤差 er と $numEpochs$ が同じであるときに、中間ノードが大きくなれば正解率が 1 になるまでのエポック数が小さくなることはすべてに当てはまった。中間ノードの値が大きくなれば分類精度が高くなっていることがわかった。しかし中間ノードの値が大きくなったとしても分類精度は変わっていない。これは前者が偶発的だった可能性がある。中間ノードの変化は分類精度と正解率が 1 になるまでのエポック数両方に変化をもたらす。具体的には中間ノードが大きくなれば分類精度が高く、エポック数が小さく、中間ノードが小さくなればその逆が起こることがわかった。

4.3 演習課題 3.1 (b)

作成した 3 層ニューラルネットワークのプログラムを 4 層ニューラルネットワーク（2 層の中間層）に拡張せよ。そして、中間層 1、2 のノード数を 50、100、200、500 などの組み合わせで変化させ、分類精度の違いについて確認せよ。また、中間層 1 層では完全に分類できなかった場合でも、中間層を 2 層にすることにより完全に分類できるようになる場合があることも確認せよ。

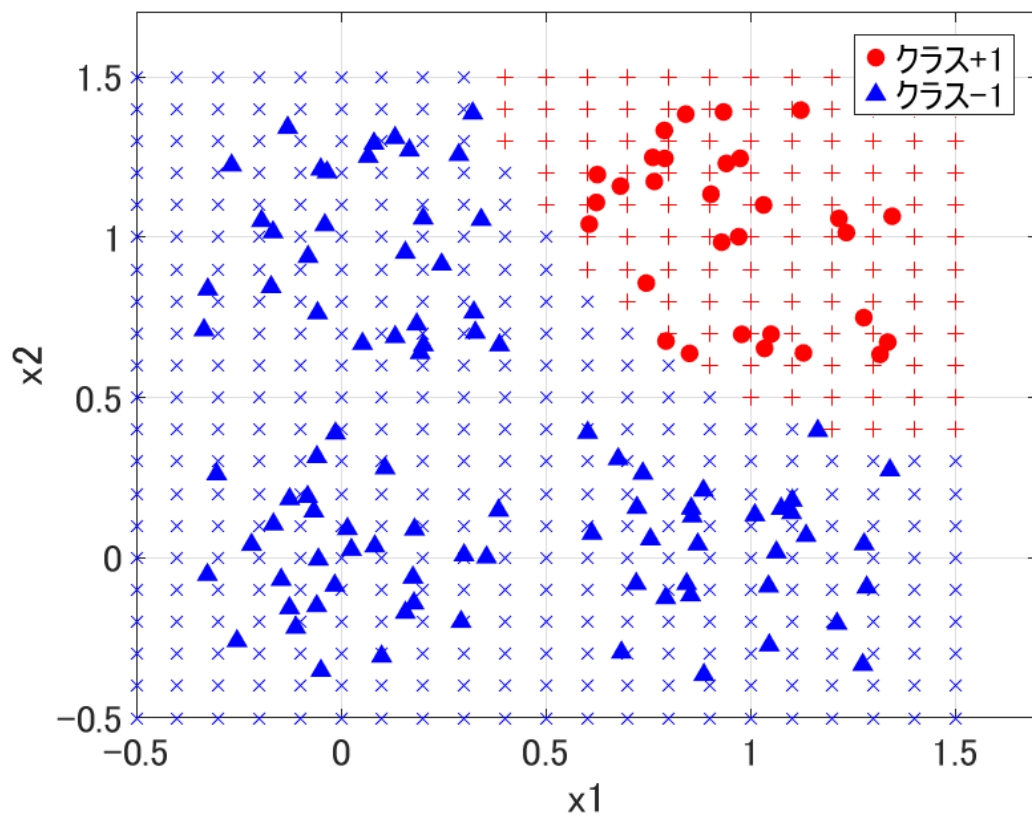


図 4.25: node=50

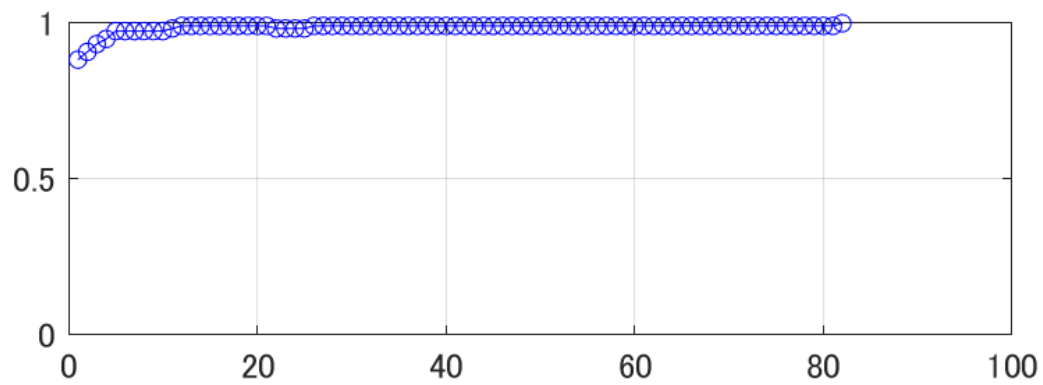


図 4.26: node=50

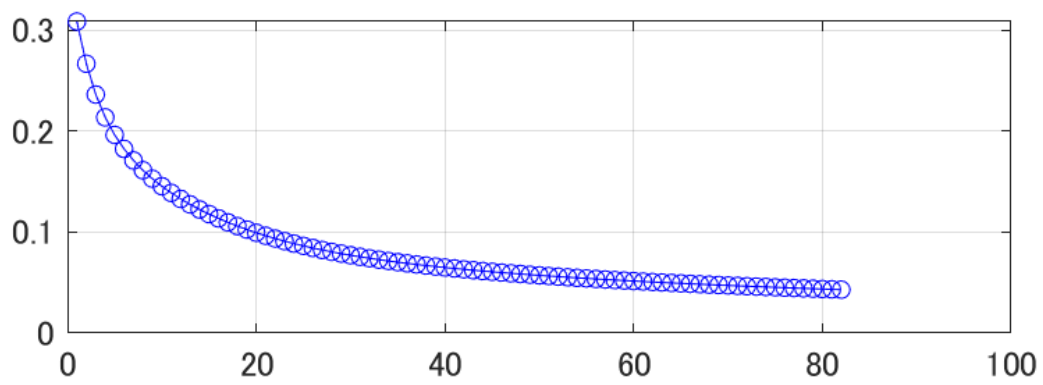


図 4.27: node=50

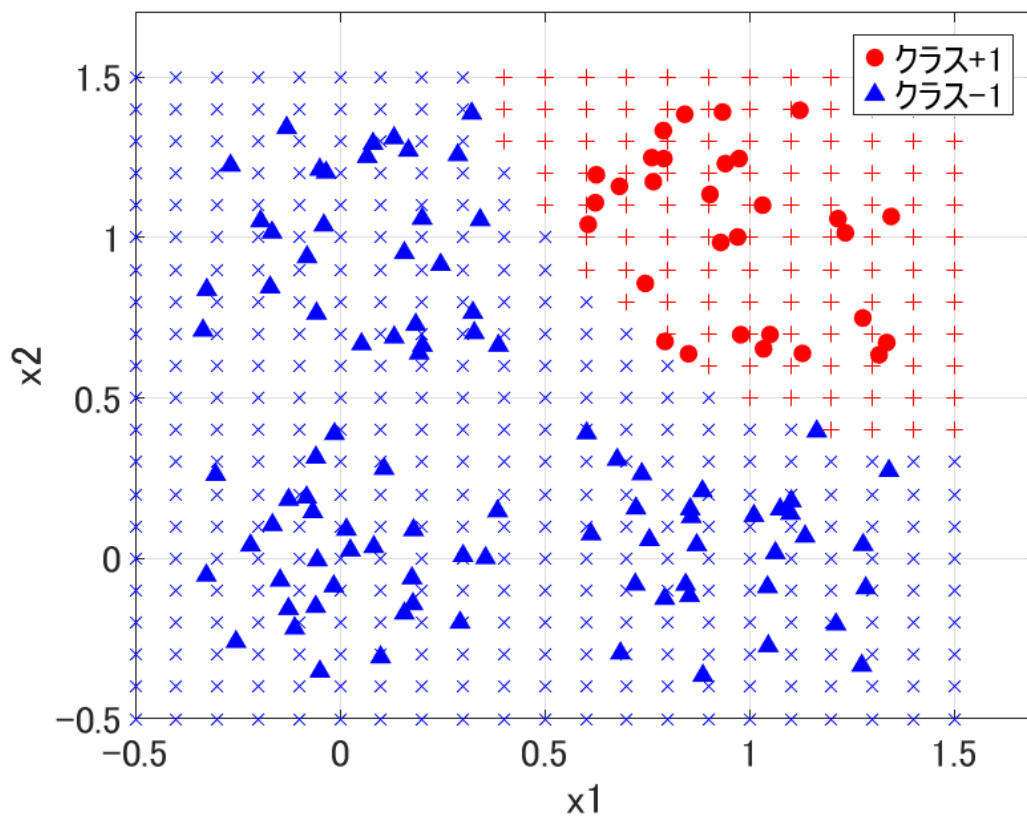


図 4.28: node=100

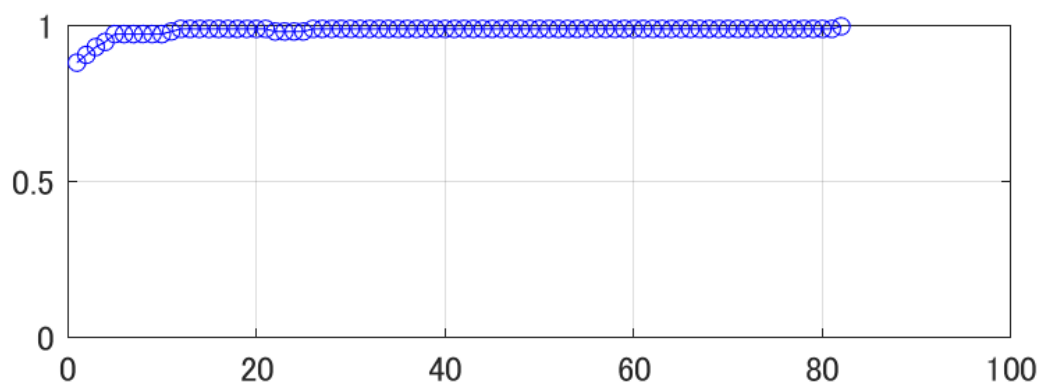


图 4.29: node=100

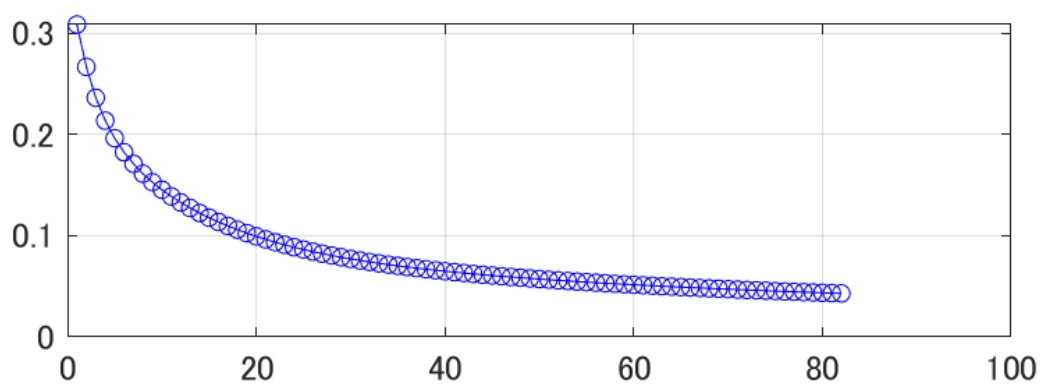


图 4.30: node=100

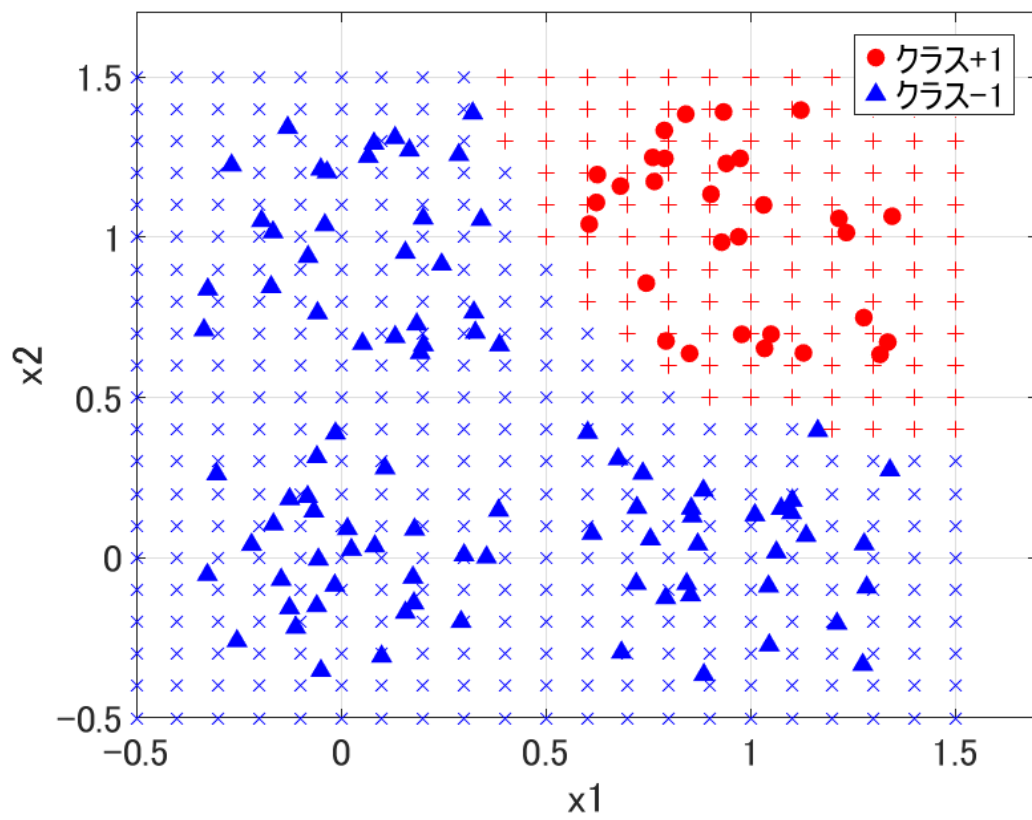


図 4.31: node=200

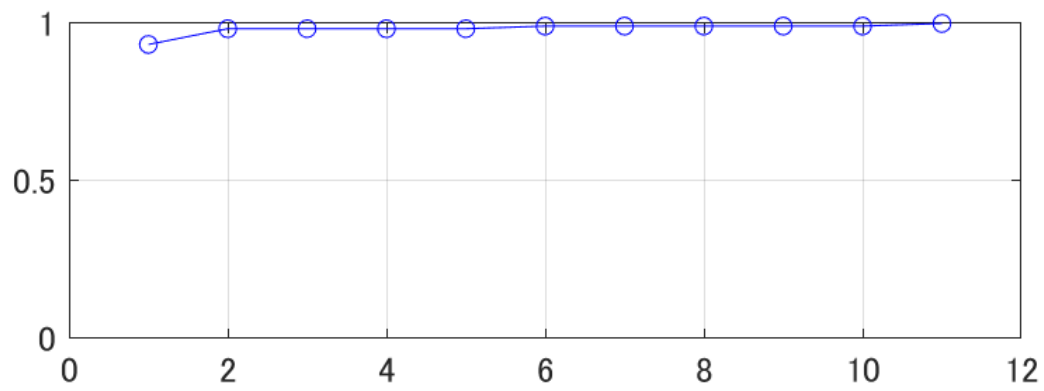


図 4.32: node=200

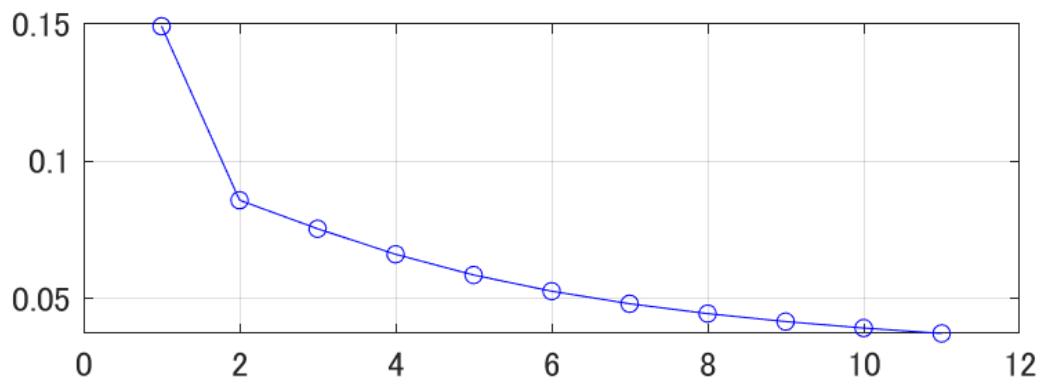


図 4.33: node=200

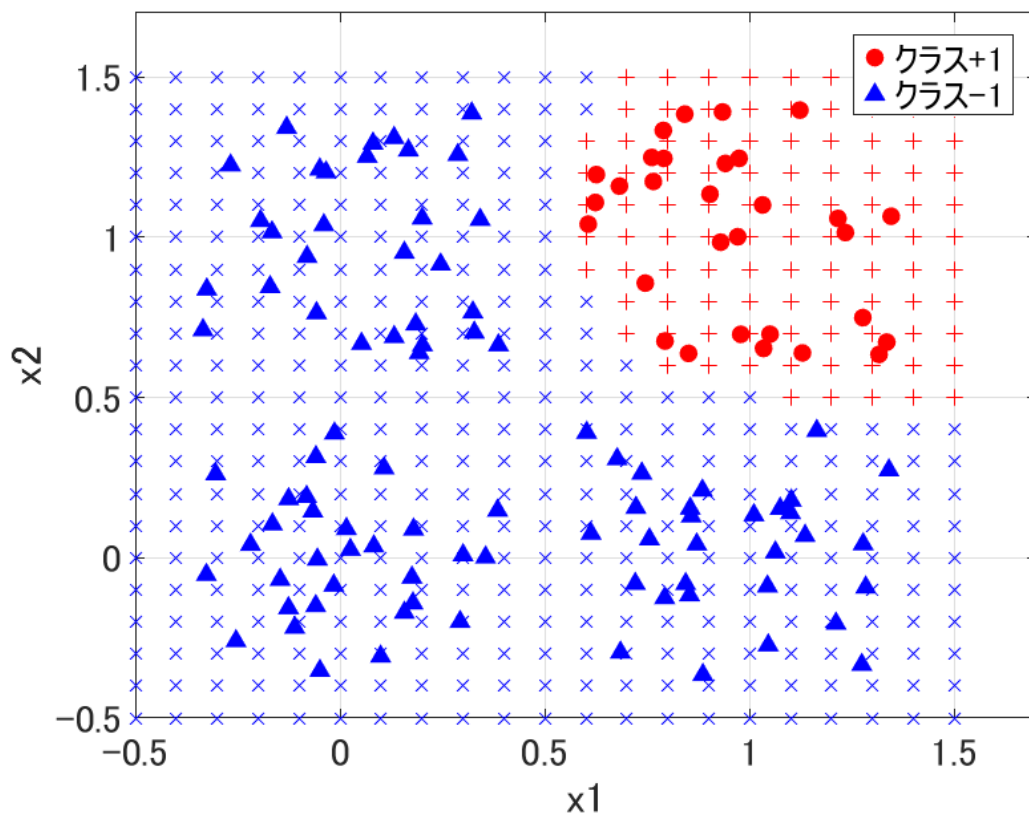


図 4.34: node=500

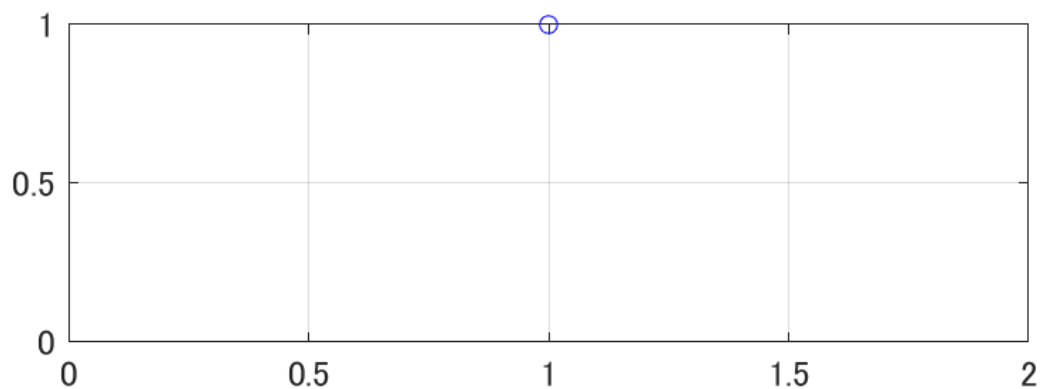


図 4.35: node=500

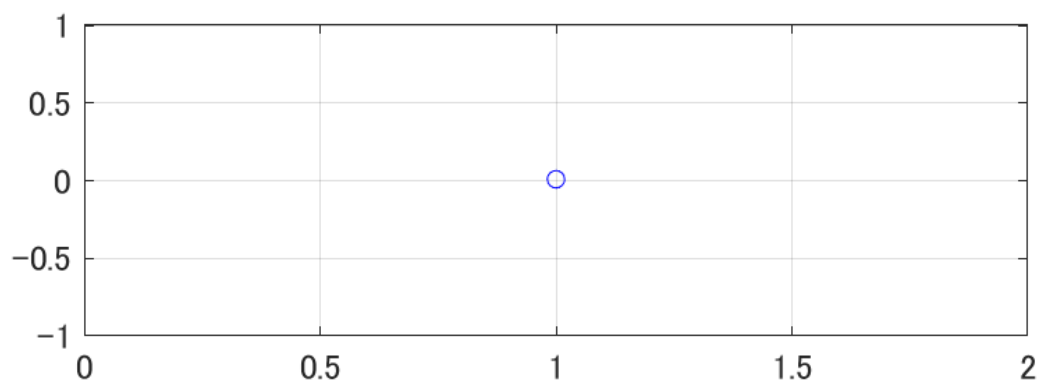


図 4.36: node=500

上記に示す複数の画像を見ると、ノード数を多くなれば

5 第4週：手書き文字数字認識への拡張

Matlab の手書き数字ベースを用い、「0、1、2、3、4、5、6、7、8、9」の10個の数字に分類する10クラス分類プログラムを作成する。

5.1 演習課題 4.1

(a) 上記のプログラムを作成し、中間層のノード数やエポック数を変化させ、テストデータの分類精度が高くなるように工夫せよ。なお、今回の学習は各エポックの計算に時間がかかるので時間の使い方に注意せよ

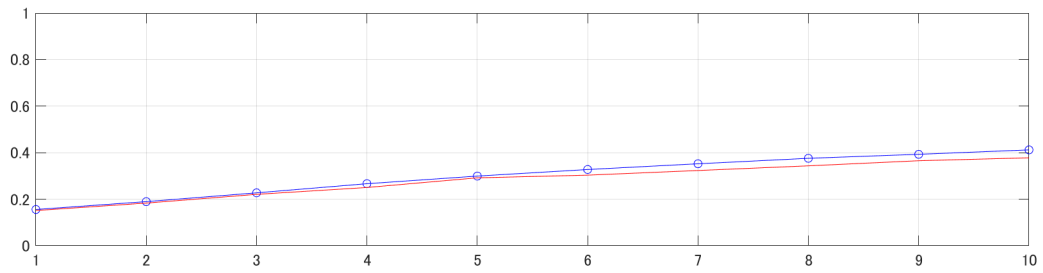


図 5.1: numEpochs=10

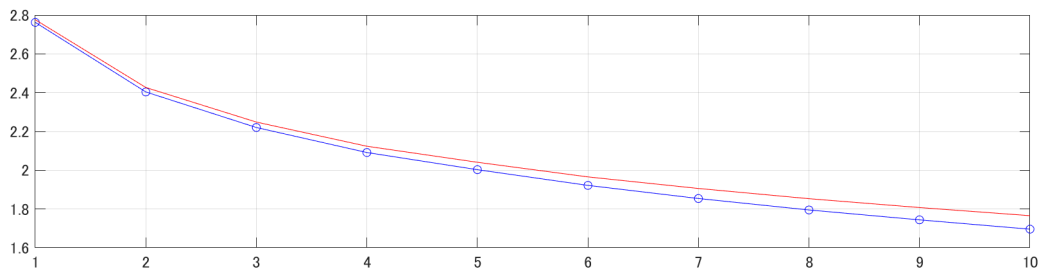


図 5.2: numEpochs=10

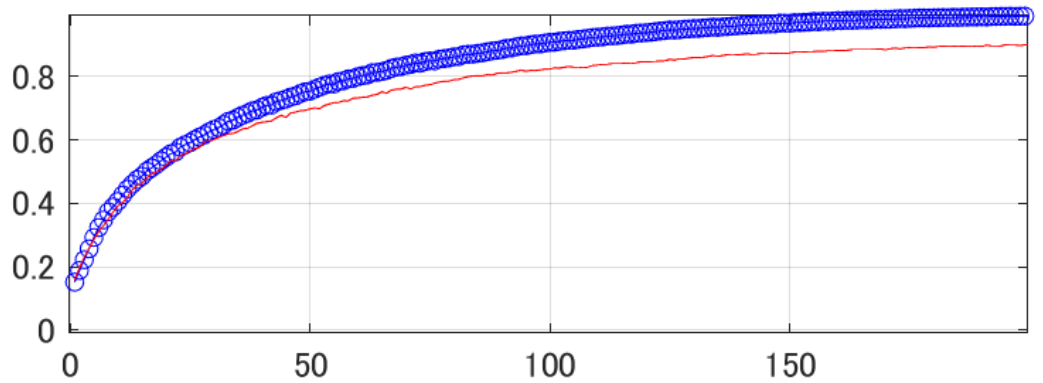


図 5.3: node=100,numEpochs=200

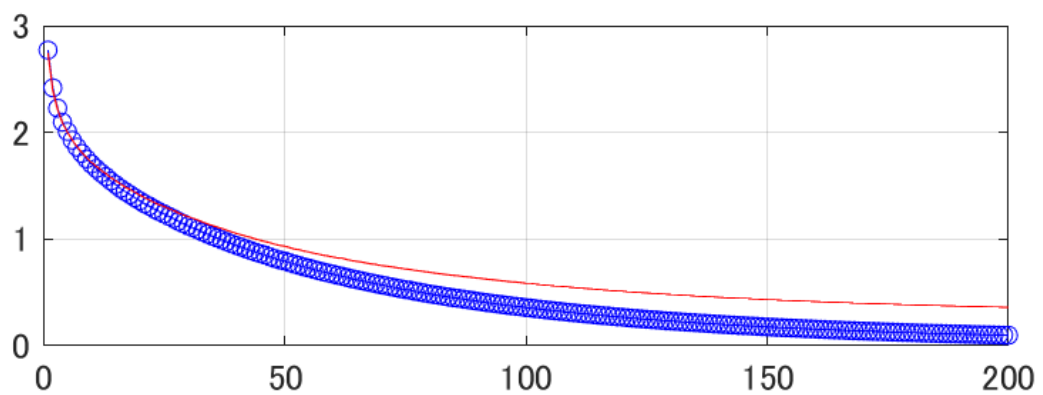


図 5.4: node=100,numEpochs=200

上記画像は、エポック数が 10 の時と 200 の時の分類精度を表したグラフである。グラフから分かるようにエポック数を増やすと分類精度を向上させることができる。

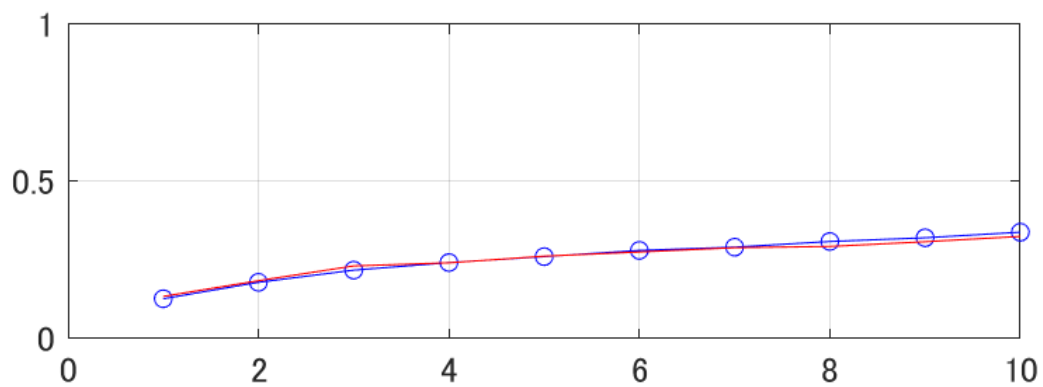


图 5.5: node1=10,node2=50,numEpochs=10

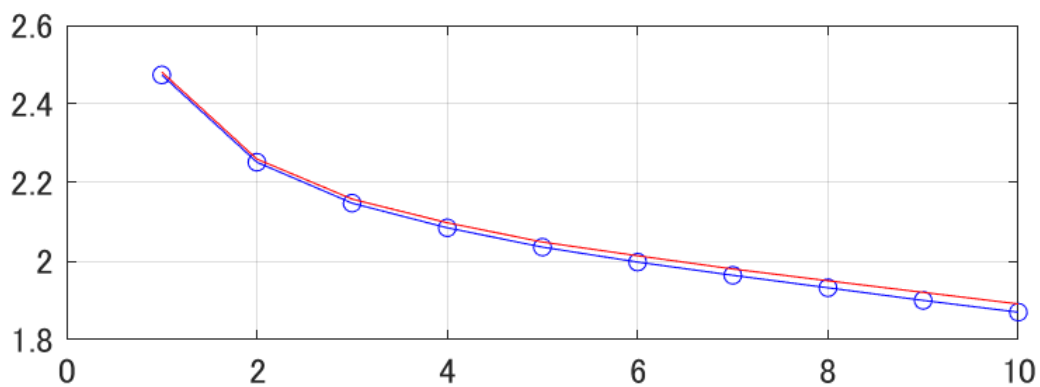


图 5.6: node1=10,node2=50,numEpochs=10

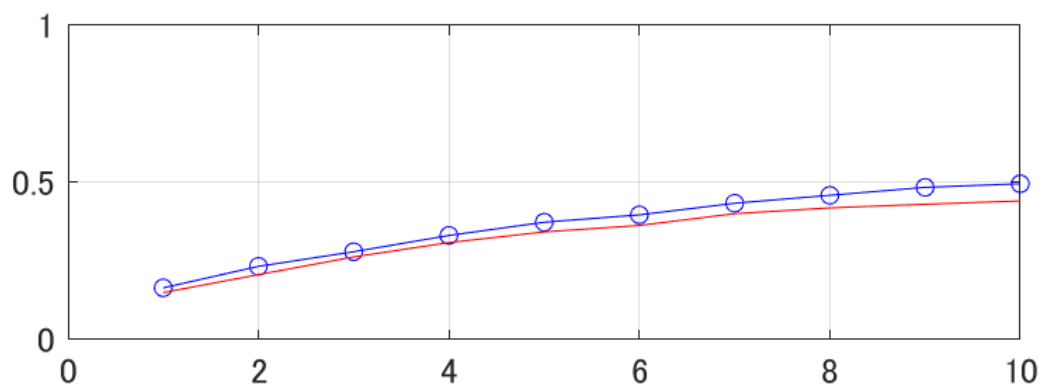


图 5.7: node1=50,node2=100,numEpochs=10

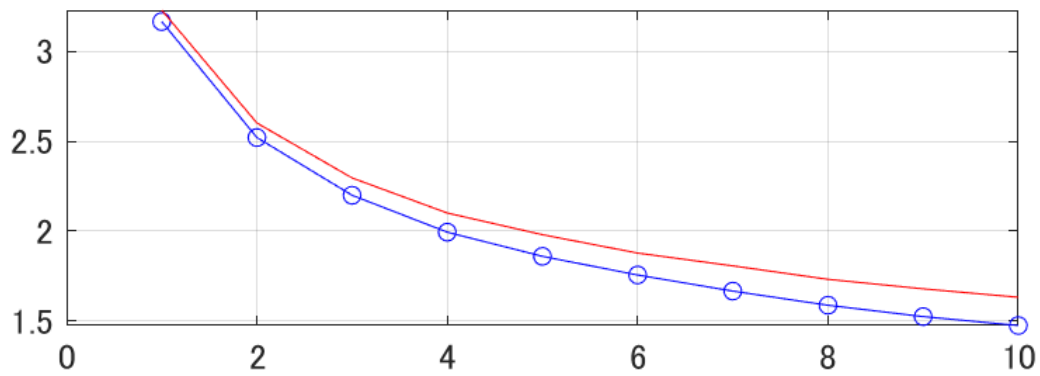


図 5.8: node1=50,node2=100,numEpochs=10

また、上画像のように、エポック数を固定してノード数を変化させると、ノード数を増やすとその分分類精度が向上していることがよく分かる。

5.2 演習課題 4.2

(a) tryPredict を実行し、様々な数字を入力し、その推定能力を確認する。

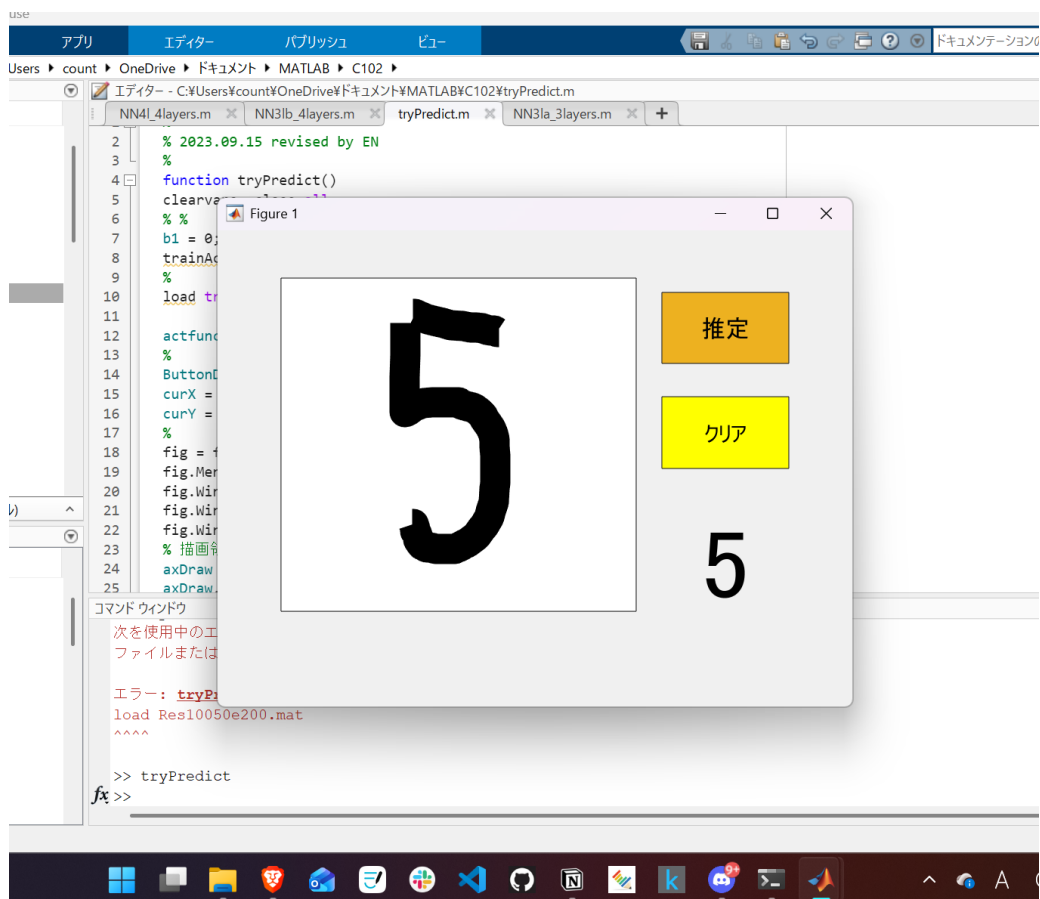


図 5.9: 手書き文字数字認識への拡張の確認

上画像のように手書き文字数字認識をすることができた。

6 ソースコードのまとめ

Listing 6.1: saikyu11

```
1 %%%% 以下を入力 %%%%
2 % プログラム1.1: saikyu11.m
3 %
4 % 変数の目的関数対する最急降下法1
5 %
6 % すべての変数の値を初期化する. グラフをすべて閉じる.
7 clearvars, close all
8 % ← (A)
9 w = 4;
10 ep = 0.01;
11 iters = 70;
12 ws = w;
13 for k = 1:iters
14     g = grad(w);
15     w = w - ep*g;
16     ws = [ws; w];
17 end
18 %
19 %% 目的関数のグラフを描く
20 % w: 変数. 範囲: から刻みでまでの範囲 -5.20.15.2
21 % Lw: 目的関数の値
22 w = -5.2:0.1:5.2;
23 Lw = obj_func(w);
24 plot(w, Lw, 'b-', 'LineWidth', 1.5), grid on, hold on
25 xlabel('w'), ylabel('L(w)')
26 set(gca, 'FontSize', 16) % フォントサイズをポイントにする. 16
27 % ← (B)
28 for k = 1:iters
29     yk = obj_func(ws(k));
30     plot([ws(k), ws(k)], [0, yk], 'rx--', 'LineWidth', 1.5);
31 end
32 %
33 %% 目的関数 (objective) の計算function
34 function y = obj_func(w)
35     y = w.^4/20 - w.^2 + w + 10;
36 end
37 % ← (C)
38 function g = grad(w)
39     g = w.^3/5 - 2*w + 1;
40 end
41 %%%% 以上を入力 %%%%
```

Listing 6.2: saikyu12

```
1 %%%% 以下を入力 %%%%
2 % プログラム1.1: saikyu11.m
3 %
4 % 変数の目的関数対する最急降下法1
5 %
```



```

6 % すべての変数の値を初期化する。グラフをすべて閉じる。
7 clearvars, close all
8 % ← (A)
9 w = 4;
10 ep = 0.1;
11 iters = 20;
12 ws = w;
13 for k = 1:iters
14     g = grad(w);
15     w = w - ep*g;
16     ws = [ws; w];
17 end
18 %
19 %% 目的関数のグラフを描く
20 % w: 変数. 範囲: から刻みでまでの範囲 -5.20.15.2
21 % Lw: 目的関数の値
22 w = -5:0.1:5;
23 Lw = obj_func(w);
24 plot(w, Lw, 'b-', 'LineWidth', 1.5), grid on, hold on
25 xlabel('w'), ylabel('L(w)')
26 set(gca, 'FontSize', 16) % フォントサイズをポイントにする. 16
27 % ← (B)
28 for k = 1:iters
29     yk = obj_func(ws(k));
30     plot([ws(k), ws(k)], [0, yk], 'rx--', 'LineWidth', 1.5);
31 end
32 %
33 %% 目的関数 (objective) の計算function
34 function y = obj_func(w)
35 y = sin(2 * w) + w.^2/8 + 2;
36 end
37 % ← (C)
38 function g = grad(w)
39 g = 2 * cos(2 * w) + w/4;
40 end
41 % 以上を入力 %

```

Listing 6.3: saikyu21

```

1 % プログラム1.2: saikyu21.m
2 %
3 % 変数の目的関数に対する最急降下法2
4 %
5 % すべての変数の値を初期化する。グラフをすべて閉じる。
6 clearvars, close all
7 % ← (A)
8 %% 最急降下法
9 % 初期値, ステップ幅, 反復回数の設定
10 wk = [1,1]; % の初期値w
11 ep = 0.1; % ステップ幅
12 iters = 20; % 反復回数 (number of ) iterations
13 wks = wk; % wks: wk のすべての値を保存する
14 % 更新の反復
15 for i=1:iters
16     gk = grad(wk(1),wk(2)); % 勾配の計算
17     wk = wk - ep*gk; % 更新
18     wks = [wks; wk]; % 反復点の保存
19 end
20
21 %% 目的関数のグラフを描く
22 % w: 変数. 範囲: から刻みでまでのメッシュ範囲 -60.256
23 % LW: 目的関数
24 [w1, w2] = meshgrid(-6:0.25:5.5);
25 LW = obj_func(w1, w2);

```

```

26 % 目的関数の描画
27 sc = meshc(w1, w2, LW); hold on
28 sc(2).LevelList = 2:2:40; % (w1, w2) 平面での等高線の間隔を指定
29 xlabel('w1'), ylabel('w2'), zlabel('L(W)')
30 set(gca, 'FontSize', 14)
31 % ← (B)
32 %% 結果の描画
33 for i=1:iters
34     yk = obj_func(wks(i,1), wks(i,2));
35     plot3([wks(i,1), wks(i,1)], [wks(i,2), wks(i,2)], [0, yk], 'rx--', 'LineWidth', 1.5)
36 end
37 %% (w1, w2) 平面での等高線の描画
38 figure % 新しい描画ウィンドウを開く
39 contour(w1, w2, LW, 25), hold on
40 plot(wks(:,1), wks(:,2), 'rx-', 'LineWidth', 1.5)
41 xlabel('w1'), ylabel('w2')
42 set(gca, 'FontSize', 14)
43 %% 目的関数の計算
44 function y = obj_func(w1, w2)
45 y = (w1.^4)/24 + (w2.^4)/32 - w1.^2 - w2.^2 + w1 + w2 + 25;
46 end
47 %
48 %% 勾配の計算
49 function g = grad(w1, w2)
50 g = [(w1.^3)/6 - 2*w1 + 1, (w2.^3)/8 - 2*w2 + 1];
51 end
52 %

```

Listing 6.4: ptron1

```

1 %%%% 以下を入力 %%%%
2 % プログラム 2.1: ptron1.m
3 % パーセプトロンの例
4 % バッチ学習
5 %
6 clearvars, close all
7 %
8 N0 = 30; % 各点付近でのデータ数
9 N = N0 * 4; % 全データ数
10 er = 0.2; % 誤差の範囲
11 %% 入力データ
12 rng(123) % 乱数発生器のシード
13 X00 = [0; 0] + rand(2, N0) * er * 2 - er; % (0, 0) 周辺のデータ
14 X10 = [1; 0] + rand(2, N0) * er * 2 - er; % (1, 0) 周辺のデータ
15 X01 = [0; 1] + rand(2, N0) * er * 2 - er; % (0, 1) 周辺のデータ
16 X11 = [1; 1] + rand(2, N0) * er * 2 - er; % (1, 1) 周辺のデータ
17 Xset = [X00, X10, X01, X11];
18 %
19 % 教師データ：クラス-1 とクラス+1 に対応し, -1 または 1 の値を持つ
20 t00 = ones(1, N0); t10 = t00; t01 = t00; t11 = t00;
21 % and 関係の場合 tset = [-t00, -t10, -t01, t11];
22 % or 関係の場合 tset = [-t00, t10, t01, t11]
23 tset = [-t00, t10, t01, t11];
24 % データセットをランダムに並べ替え
25 p = randperm(N);
26 Xset = Xset(:, p); % 入力データ
27 tset = tset(:, p); % 教師データ
28 % ← (A)
29 %%%% 以下を入力 %%%%
30 %% 学習
31 % 初期値, 学習率, エポック数の設定
32 W = [0.5, -0.5]; b = 0;
33 ep = 0.1;
34 epochs = 200;

```

```

35 Ws = W; bs = b; % 学習パラメータ W, の保存b
36 LWbs = []; % 損失関数の値の保存
37 accs = []; % 正解率の保存
38 % 最急降下法
39 for k=1:epochs
40     LWb = 0;
41     nCorrect = 0;
42     gW = [0, 0]; gb = 0;
43     for j=1:N
44         LWbj = lossFunc(W, b, Xset(:, j), tset(j));
45         LWb = LWb + LWbj;
46         if LWbj==0
47             nCorrect = nCorrect + 1;
48         else
49             [gWj, gb] = grad(Xset(:, j), tset(j));
50             gW = gW + gWj; gb = gb + gb;
51         end
52     end
53     LWbs = [LWbs, LWb/N];
54     accs = [accs, nCorrect/N];
55     gW = gW/N; gb = gb/N;
56     if nCorrect==N
57         break; % すべてのデータの分類ができたなら反復を終了する.
58     end
59     W = W - ep*gW; b = b - ep*gb;
60     Ws = [Ws; W]; bs = [bs; b];
61 end
62 %
63 %%%%% 以上を入力 %%%%%
64 %% 入力データの描画
65 for i=1:length(tset)
66     if tset(i)>0
67         p1 = plot(Xset(1, i), Xset(2, i), 'ro', 'MarkerFaceColor','r'); hold on
68     else
69         p2 = plot(Xset(1, i), Xset(2, i), 'b^', 'MarkerFaceColor','b'); hold on
70     end
71 end
72 axis([-0.5, 1.7, -0.5, 1.7]), grid on
73 xlabel('x1'), ylabel('x2')
74 legend([p1, p2], 'クラス+1', 'クラス-1');
75 set(gca, 'FontSize', 14)
76 %
77 %%%%% 以下を入力 %%%%%
78 %% 結果の描画
79 % 学習したパラメータから求めた直線の描画
80 w1 = Ws(end, 1); w2 = Ws(end, 2); b = bs(end);
81 x1s = [-0.5, 1.5];
82 x2s = -(w1*x1s + b)/w2;
83 p3 = plot(x1s, x2s, 'g', 'LineWidth', 1.0);
84 % 初期値から求めた直線の描画
85 w1 = Ws(1, 1); w2 = Ws(1, 2); b = bs(1);
86 x2s = -(w1*x1s + b)/w2;
87 p4 = plot(x1s, x2s, 'g--', 'LineWidth', 1.0);
88 legend([p1, p2, p3, p4], 'クラス+1', 'クラス-1', '結果の直線', '初期直線')
89 % 損失関数値, 正解率の描画
90 figure
91 plot(LWbs, 'b', 'LineWidth', 1.0), hold on, grid on
92 plot(accs, 'r', 'LineWidth', 1.0)
93 legend('損失関数値', '正解率')
94 xlabel('エポック数')
95 set(gca, 'FontSize', 14)
96 %%%%% 以上を入力 %%%%%
97 % ← (B)
98 %% 一つのに対する損失関数の計算data

```

```

99 function L = lossFunc(W, b, X, t)
100 L = max(0, -t * (W * X + b));
101 end
102 %
103 %% 一つのに対する損失関数の勾配data
104 function [gLW, gLb] = grad(X, t)
105 gLW = -t * X';
106 gLb = -t;
107 end

```

Listing 6.5: patron2

```

1  %%%%% 以下を入力 %%%%%
2  % プログラム 2.1: ptron2.m
3  % パーセプトロンの例
4  % オンライン学習
5  %
6  clearvars, close all
7  %
8  N0 = 30; % 各点付近でのデータ数
9  N = N0 * 4; % 全データ数
10 er = 0.2; % 誤差の範囲
11 %% 入力データ
12 rng(123) % 乱数発生器のシード
13 X00 = [0; 0] + rand(2, N0) * er * 2 - er; % (0, 0) 周辺のデータ
14 X10 = [1; 0] + rand(2, N0) * er * 2 - er; % (1, 0) 周辺のデータ
15 X01 = [0; 1] + rand(2, N0) * er * 2 - er; % (0, 1) 周辺のデータ
16 X11 = [1; 1] + rand(2, N0) * er * 2 - er; % (1, 1) 周辺のデータ
17 Xset = [X00, X10, X01, X11];
18 %
19 % 教師データ: クラス-1 とクラス+1 に対応し, -1 または 1 の値を持つ
20 t00 = ones(1, N0); t10 = t00; t01 = t00; t11 = t00;
21 % and 関係の場合 tset = [-t00, -t10, -t01, t11];
22 % or 関係の場合 tset = [-t00, t10, t01, t11]
23 tset = [-t00, -t10, -t01, t11];
24 % データセットをランダムに並べ替え
25 p = randperm(N);
26 Xset = Xset(:, p); % 入力データ
27 tset = tset(:, p); % 教師データ
28 % ← (A)
29 %%%%% 以下を入力 %%%%%
30 %% 学習
31 % 初期値, 学習率, エポック数の設定
32 W = [1, -1]; b = 0;
33 ep = 0.1;
34 epochs = 200;
35 Ws = W; bs = b; % 学習パラメータ W, の保存b
36 LWbs = []; % 損失関数の値の保存
37 accs = []; % 正解率の保存
38 % 最急降下法
39 for k=1:epochs
40     LWb = 0;
41     nCorrect = 0;
42     gW = [0, 0]; gb = 0;
43     for j=1:N
44         LWbj = lossFunc(W, b, Xset(:, j), tset(j));
45         LWb = LWb + LWbj;
46         if LWbj==0
47             nCorrect = nCorrect + 1;
48         else
49             [gWj, gb] = grad(Xset(:, j), tset(j));
50             W = W - ep*gWj; b = b - ep*gb;
51         end
52     end

```

```

53 LWbs = [LWbs, LWb/N];
54 accs = [accs, nCorrect/N];
55 gW = gW/N; gb = gb/N;
56 if nCorrect==N
57     break; % すべてのデータの分類ができたなら反復を終了する.
58 end
59 Ws = [Ws; W]; bs = [bs; b];
60 end
61 %
62 %%%%% 以上を入力 %%%%%
63 %% 入力データの描画
64 for i=1:length(tset)
65     if tset(i)>0
66         p1 = plot(Xset(1, i), Xset(2, i), 'ro', 'MarkerFaceColor','r'); hold on
67     else
68         p2 = plot(Xset(1, i), Xset(2, i), 'b^', 'MarkerFaceColor','b'); hold on
69     end
70 end
71 axis([-0.5, 1.7, -0.5, 1.7]), grid on
72 xlabel('x1'), ylabel('x2')
73 legend([p1, p2], 'クラス+1', 'クラス-1');
74 set(gca, 'FontSize', 14)
75 %
76 %%%%% 以下を入力 %%%%%
77 %% 結果の描画
78 % 学習したパラメータから求めた直線の描画
79 w1 = Ws(end, 1); w2 = Ws(end, 2); b = bs(end);
80 x1s = [-0.5, 1.5];
81 x2s = -(w1*x1s + b)/w2;
82 p3 = plot(x1s, x2s, 'g', 'LineWidth', 1.0);
83 % 初期値から求めた直線の描画
84 w1 = Ws(1, 1); w2 = Ws(1, 2); b = bs(1);
85 x2s = -(w1*x1s + b)/w2;
86 p4 = plot(x1s, x2s, 'g--', 'LineWidth', 1.0);
87 legend([p1, p2, p3, p4], 'クラス+1', 'クラス-1', '結果の直線', '初期直線')
88 % 損失関数値, 正解率の描画
89 figure
90 plot(LWbs, 'b', 'LineWidth', 1.0), hold on, grid on
91 plot(accs, 'r', 'LineWidth', 1.0)
92 legend('損失関数値', '正解率')
93 xlabel('エポック数')
94 set(gca, 'FontSize', 14)
95 %%%%% 以上を入力 %%%%%
96 % ← (B)
97 %% 一つのに対する損失関数の計算data
98 function L = lossFunc(W, b, X, t)
99     L = max(0, -t * (W * X + b));
100 end
101 %
102 %% 一つのに対する損失関数の勾配data
103 function [gLW, gLb] = grad(X, t)
104     gLW = -t * X';
105     gLb = -t;
106 end

```

Listing 6.6: ptron3

```

1 %%%%% 以下を入力 %%%%%
2 % プログラム 2.1: ptron3.m
3 % パーセプトロン の例
4 % ミニバッチ学習
5 %
6 clearvars, close all
7 %

```

```

8 N0 = 30; % 各点付近でのデータ数
9 N = N0 * 4; % 全データ数
10 er = 0.2; % 誤差の範囲
11 %% 入力データ
12 rng(123) % 乱数発生器のシード
13 X00 = [0; 0] + rand(2, N0) * er * 2 - er; % (0, 0) 周辺のデータ
14 X10 = [1; 0] + rand(2, N0) * er * 2 - er; % (1, 0) 周辺のデータ
15 X01 = [0; 1] + rand(2, N0) * er * 2 - er; % (0, 1) 周辺のデータ
16 X11 = [1; 1] + rand(2, N0) * er * 2 - er; % (1, 1) 周辺のデータ
17 Xset = [X00, X10, X01, X11];
18 %
19 % 教師データ：クラス-1 とクラス+1 に対応し、-1 または 1 の値を持つ
20 t00 = ones(1, N0); t10 = t00; t01 = t00; t11 = t00;
21 % and 関係の場合 tset = [-t00, -t10, -t01, t11];
22 % or 関係の場合 tset = [-t00, t10, t01, t11]
23 tset = [-t00, -t10, -t01, t11];
24 % データセットをランダムに並べ替え
25 p = randperm(N);
26 Xset = Xset(:, p); % 入力データ
27 tset = tset(:, p); % 教師データ
28 % ← (A)
29 %%%% 以下を入力 %%%%
30 %% 学習
31 % 初期値, 学習率, エポック数の設定
32 W = [1, -1]; b = 0;
33 ep = 0.1;
34 epochs = 200;
35 Ws = W; bs = b; % 学習パラメータ W, の保存b
36 LWbs = []; % 損失関数の値の保存
37 accs = []; % 正解率の保存
38 M = 4;
39 ng = N / M;
40 % 最急降下法
41 for k=1:epochs
42     LWb = 0;
43     nCorrect = 0;
44     gW = [0, 0]; gb = 0;
45     for i=1:ng
46         j = (i - 1) * M + 1 : i * M
47         LWbj = lossFunc(W, b, Xset(:, j), tset(j));
48         LWb = LWb + LWbj;
49         if LWbj==0
50             nCorrect = nCorrect + 1;
51         else
52             [gWj, gb] = grad(Xset(:, j), tset(j));
53             gW = gW + gWj; gb = gb + gb;
54         end
55     end
56     gW = gW/M; gb = gb/M;
57     W = W - ep*gW; b = b - ep*gb;
58     end
59     LWbs = [LWbs, LWb/N];
60     accs = [accs, nCorrect/N];
61     if nCorrect==N
62         break; % すべてのデータの分類ができたなら反復を終了する.
63     end
64     Ws = [Ws; W]; bs = [bs; b];
65 end
66 %
67 %%%% 以上を入力 %%%%
68 %% 入力データの描画
69 for i=1:length(tset)
70     if tset(i)>0
71         p1 = plot(Xset(1, i), Xset(2, i), 'ro', 'MarkerFaceColor','r'); hold on

```

```

72     else
73     p2 = plot(Xset(1, i), Xset(2, i), 'b^', 'MarkerFaceColor','b'); hold on
74     end
75 end
76 axis([-0.5, 1.7, -0.5, 1.7]), grid on
77 xlabel('x1'), ylabel('x2')
78 legend([p1, p2], 'クラス+1', 'クラス-1');
79 set(gca, 'FontSize', 14)
80 %
81 %%%% 以下を入力 %%%%
82 %% 結果の描画
83 % 学習したパラメータから求めた直線の描画
84 w1 = Ws(end, 1); w2 = Ws(end, 2); b = bs(end);
85 x1s = [-0.5, 1.5];
86 x2s = -(w1*x1s + b)/w2;
87 p3 = plot(x1s, x2s, 'g', 'LineWidth', 1.0);
88 % 初期値から求めた直線の描画
89 w1 = Ws(1, 1); w2 = Ws(1, 2); b = bs(1);
90 x2s = -(w1*x1s + b)/w2;
91 p4 = plot(x1s, x2s, 'g--', 'LineWidth', 1.0);
92 legend([p1, p2, p3, p4], 'クラス+1', 'クラス-1', '結果の直線', '初期直線')
93 % 損失関数値, 正解率の描画
94 figure
95 plot(LWbs, 'b', 'LineWidth', 1.0), hold on, grid on
96 plot(accs, 'r', 'LineWidth', 1.0)
97 legend('損失関数値', '正解率')
98 xlabel('エポック数')
99 set(gca, 'FontSize', 14)
100 %%%% 以上を入力 %%%%
101 % ← (B)
102 %% 一つのに対する損失関数の計算data
103 function L = lossFunc(W, b, X, t)
104 L = max(0, -t * (W * X + b));
105 end
106 %
107 %% 一つのに対する損失関数の勾配data
108 function [gLW, gLb] = grad(X, t)
109 gLW = -t * X';
110 gLb = -t;
111 end

```

Listing 6.7: NN3la3layers

```

1 %%%% 以下を入力 %%%%
2 % プログラム 3.1: NN1mlayer31a.m
3 % 中間層を 1 層持つニューラルネットワーク
4 % ミニバッチ学習
5 %
6 clearvars, close all
7 % 大域変数の定義. 後に定義する関数と共有する変数を指定.
8 global W1 W2 b1 b2
9 global dW1 dW2 db1 db2
10 %
11 %% 各種設定
12 % 入力データの設定
13 N0 = 30; % 各点付近でのデータ数
14 N = N0*4; % 全データ数
15 er = 0.3; % 誤差の範囲
16 % エポック数, バッチサイズ, 学習率
17 numEpochs = 100;
18 batchSize = 20;
19 ep = 0.1;
20 %
21 %% 入力データ

```

```

22 rng(123) % 乱数発生器のシード
23 X00 = [0; 0] + rand(2, N0)*er*2 - er; % (0, 0)' 周辺のデータ
24 X10 = [1; 0] + rand(2, N0)*er*2 - er; % (1, 0)' "
25 X01 = [0; 1] + rand(2, N0)*er*2 - er; % (0, 1)' "
26 X11 = [1; 1] + rand(2, N0)*er*2 - er; % (1, 1)' "
27 Xset = [X00, X10, X01, X11];
28 % 教師データ：クラス-1 とクラス+1 に対応し, -1 または 1 の値を持つ
29 t00 = ones(1, N0); t10 = t00; t01 = t00; t11 = t00;
30 % and 関係の場合 tset = [-t00, -t10, -t01, t11];
31 % or 関係の場合 tset = [-t00, t10, t01, t11]
32 tset = [-t00, -t10, -t01, t11];
33 % データセットをランダムに並べ替え
34 p = randperm(N);
35 Xset = Xset(:, p); % 入力データ
36 tset = tset(:, p); % 教師データ
37 % ← (A)
38 dimInput = 2; % 入力の次元 () dimension
39 dimLayer1 = 50; % 中間層のノード数1
40 dimOutput = 2; % 出力の次元
41 rng(123) % 乱数のシード指定
42 % 重みとバイアスの初期設定. 乱数値を入力.
43 W1 = randn(dimLayer1, dimInput);
44 W2 = randn(dimOutput, dimLayer1);
45 b1 = randn(dimLayer1, 1);
46 b2 = randn(dimOutput, 1);
47 % 重みとバイアスの初期設定
48 dW1 = zeros(size(W1)); dW2 = zeros(size(W2));
49 db1 = zeros(size(b1)); db2 = zeros(size(b2));
50 % 勾配の値を入れる変数の定義
51 gW1 = zeros(size(W1)); gW2 = zeros(size(W2));
52 gb1 = zeros(size(b1)); gb2 = zeros(size(b2));
53
54 %% 学習の実行
55 numIters = N/batchSize; % エポックにおけるミニバッチの反復回数1
56 trainAccuracy = zeros(1, numEpochs);
57 trainLoss = zeros(1, numEpochs);
58 %
59 for k = 1:numEpochs
60     str = sprintf('エポック %d', k);
61     disp(str)
62     for i = 1:numIters
63         % 勾配の初期化
64         gW1 = gW1*0; gW2 = gW2*0;
65         gb1 = gb1*0; gb2 = gb2*0;
66         for j = 1:batchSize
67             j1 = batchSize*(i-1) + j;
68             grad(Xset(:, j1), tc2T(tset(j1)));
69             gW1 = gW1 + dW1; gW2 = gW2 + dW2;
70             gb1 = gb1 + db1; gb2 = gb2 + db2;
71         end
72         gW1 = gW1/batchSize; gW2 = gW2/batchSize;
73         gb1 = gb1/batchSize; gb2 = gb2/batchSize;
74         W1 = W1 - ep*gW1; W2 = W2 - ep*gW2;
75         b1 = b1 - ep*gb1; b2 = b2 - ep*gb2;
76     end
77     % ← (C)
78     % 分類精度とクロス (交差) エントロピー誤差の計算
79     nCorrect = 0; loss = 0;
80     for j = 1:N
81         [yc, Y] = predict(Xset(:, j));
82         loss = loss + cross_entropy(softmax(Y), tc2T(tset(j)));
83         if yc==tset(j)
84             nCorrect = nCorrect + 1;
85     end

```



```

86     end
87     trainAccuracy(k) = nCorrect/N;
88     trainLoss(k) = loss/N;
89     subplot(211), plot(trainAccuracy(1:k), 'bo-'), hold on, grid on
90     ylim([0,1]), set(gca, 'FontSize', 12)
91     subplot(212), plot(trainLoss(1:k), 'bo-'), hold on, grid on
92     set(gca, 'FontSize', 12)
93     drawnow
94     if nCorrect==N
95         break;
96     end
97 end
98 %% 入力データの描画
99 figure
100 for i=1:length(tset)
101     if tset(i)>0
102         p1 = plot(Xset(1, i), Xset(2, i), 'ro', 'MarkerFaceColor','r'); hold on
103     else
104         p2 = plot(Xset(1, i), Xset(2, i), 'b^', 'MarkerFaceColor','b'); hold on
105     end
106 end
107 axis([-0.5, 1.7, -0.5, 1.7]), grid on
108 xlabel('x1'), ylabel('x2')
109 % legend([p1, p2], クラス'+1', クラス'-1');
110 set(gca, 'FontSize', 14)
111
112 % ← (B)
113 [X1, X2] = meshgrid(-0.5:0.1:1.5);
114 for i=1:size(X1, 1)
115     for j=1:size(X1, 2)
116         Xij = [X1(i, j); X2(i, j)];
117         yc = predict(Xij);
118         if yc<0
119             plot(X1(i, j), X2(i, j), 'bx')
120         else
121             plot(X1(i, j), X2(i, j), 'r+')
122         end
123     end
124 end
125 legend([p1, p2], 'クラス+1', 'クラス-1');
126 %
127 %% スカラー教師データのベクトル化
128 function T = tc2T(tc)
129 T = zeros(2, 1); % 2行のゼロベクトルx1
130 if tc== -1
131     T(1) = 1; % クラスの場合, 第要素が-111
132 else
133     T(2) = 1; % クラスの場合, 第要素が+122
134 end
135 end
136 %% クラス予測とソフトマックス関数入力前の出力NN
137 function [yc, Y] = predict(X)
138 global W1 W2 b1 b2
139 global Z1
140 Z1 = sigmoid(W1*X + b1);
141 Y = W2*Z1 + b2;
142 [~, ind] = max(Y);
143 if ind==1
144     yc = -1;
145 else
146     yc = 1;
147 end
148 end
149 %

```

```

150 %% シグモイド関数
151 function Y = sigmoid(Z)
152 Y = 1./(1 + exp(-Z));
153 end
154 %
155 %% ソフトマックス関数
156 function Y = softmax(Z)
157 c = max(Z);
158 exp_xc = exp(Z - c);
159 sum_exp_xc = sum(exp_xc);
160 Y = exp_xc/sum_exp_xc;
161 end
162 %
163 %% 一つのデータに対するクロス（交差）エントロピー誤差
164 function Lf = cross_entropy(Y, T)
165 c = 1.0e-7;
166 Lf = -sum(T.*log(Y + c));
167 end
168 %
169 %% 誤差逆伝搬法を用いた勾配の計算
170 function grad(X, T)
171 global W2
172 global dW1 dW2 db1 db2
173 global Z1
174 % のソフトマックス出力の計算NN
175 [~, Y] = predict(X);
176 Y = softmax(Y);
177 % 勾配 db2, dW2 の計算
178 db2 = Y - T;
179 dW2 = db2*Z1';
180 % 勾配 db1, dW1 の計算
181 qZ1 = Z1.*(1 - Z1);
182 db1 = W2'*db2.*qZ1;
183 dW1 = db1*X';
184 end
185 %
186 %%%% 以上を入力 %%%%

```

Listing 6.8: NN3lb4layers

```

1 %%%% 以下を入力 %%%%
2 % プログラム3.1: NN1mlayer31.m
3 % 中間層を属持つニューラルネットワーク1
4 % ミニバッチ学習
5 %
6 clearvars, close all
7 % 大域変数の定義。後に定義する関数と共有する変数を指定。
8 global W1 W2 W3 b1 b2 b3
9 global dW1 dW2 dW3 db1 db2 db3
10 %
11 N0 = 30; % 各点付近でのデータ数
12 N = N0*4; % 全データ数
13 er = 0.4; % 誤差の範囲
14 %% 入力データ
15 rng(123) % 乱数発生器のシード
16 X1 = [0; 0] + rand(2, N0)*er*2 - er; % (0, 0)' 周辺のデータ
17 X2 = [1; 0] + rand(2, N0)*er*2 - er; % (1, 0)' "
18 X3 = [0; 1] + rand(2, N0)*er*2 - er; % (0, 1)' "
19 X4 = [1; 1] + rand(2, N0)*er*2 - er; % (1, 1)' "
20 X0 = [X1, X2, X3, X4];
21 %
22 % 教師データ：クラスとクラスに対応し、またはの値を持つ-1+1-11
23 T1 = ones(1, N0); T2 = T1; T3 = T1; T4 = T1;
24 % and 関係の場合 T0 = [-T1, -T2, -T3, T4];

```

```

25 % or 関係の場合 T0 = [-T1, T2, T3, T4]
26 T0 = [-T1, -T2, -T3, T4];
27 % データセットをランダムに並べ替え
28 p = randperm(N);
29 X = X0(:, p); % 入力データ
30 T = T0(:, p); % 教師データ
31 %% の初期設定NN
32 dimInput = 2; % 入力の次元 () dimension
33 dimLayer1 = 500; % 中間層のノード数1
34 dimLayer2 = 500;
35 dimOutput = 2; % 出力の次元
36 rng(123) % 乱数のシード指定
37 % 重みとバイアスの初期設定. 乱数値を入力.
38 W1 = randn(dimLayer1, dimInput);
39 W2 = randn(dimLayer2, dimLayer1);
40 W3 = randn(dimOutput, dimLayer2);
41 b1 = randn(dimLayer1, 1);
42 b2 = randn(dimLayer2, 1);
43 b3 = randn(dimOutput, 1);
44 % 重みとバイアスの初期設定
45 dW1 = zeros(size(W1));
46 dW2 = zeros(size(W2));
47 dW3 = zeros(size(W3));
48 db1 = zeros(size(b1));
49 db2 = zeros(size(b2));
50 db3 = zeros(size(b3));
51 % エポック数, ミニバッチのバッチサイズ, 学習率
52 numEpochs = 100;
53 batchSize = 20;
54 ep = 0.1;
55 % 以上を入力 %
56 % 以下を入力 %
57 % 学習
58 % 反復回数, 精度とクロスエントロピー誤差の結果の保存用
59 numIters = N/batchSize; % エポックにおけるミニバッチの反復数1
60 trainAccuracy = zeros(1, numEpochs);
61 trainLoss = zeros(1, numEpochs);
62 % 勾配の値を入れる変数の定義
63 gW1 = zeros(size(W1));
64 gW2 = zeros(size(W2));
65 gW3 = zeros(size(W3));
66 gb1 = zeros(size(b1));
67 gb2 = zeros(size(b2));
68 gb3 = zeros(size(b3));
69 %
70 for i = 1:numEpochs
71     str = sprintf('エポック %d', i);
72     disp(str)
73     for j = 1:numIters
74         for k = 1:batchSize
75             k1 = batchSize*(j - 1) + k;
76             Xell = X(:, k1);
77             tell = T(k1);
78             if tell==1
79                 Tell = [0; 1]; % tellの場合=1
80             else
81                 Tell = [1; 0]; % tellの場合=-1
82             end
83             ySmax = predictSmax(Xell);
84             grad(Xell, ySmax, Tell);
85             gW1 = gW1 + dW1; gW2 = gW2 + dW2; gW3 = gW3 + dW3;
86             gb1 = gb1 + db1; gb2 = gb2 + db2; gb3 = gb3 + db3;
87         end
88     gW1 = gW1/batchSize; gW2 = gW2/batchSize; gW3 = gW3/batchSize;

```

```

89 gb1 = gb1/batchSize; gb2 = gb2/batchSize; gb3 = gb3/batchSize;
90 W1 = W1 - ep*gW1; W2 = W2 - ep*gW2; W3 = W3 - ep*gW3;
91 b1 = b1 - ep*gb1; b2 = b2 - ep*gb2; b3 = b3 - ep*gb3;
92 % 次の反復のための勾配の初期化
93 gW1 = gW1*0; gW2 = gW2*0; gW3 = gW3*0;
94 gb1 = gb1*0; gb2 = gb2*0; gb3 = gb3*0;
95 end % ← (C)
96 %%%%%% 以下を入力 %%%%%%
97 % 分類精度とクロスエントロピー誤差の計算
98 nCorrect = 0; loss = 0;
99 for j = 1:N
100 ySmax = predictSmax(X(:, j));
101 [~, ym] = max(ySmax);
102 tell = T(j);
103 if tell==1
104 Tell = [0; 1];
105 else
106 Tell = [1; 0];
107 end
108 loss = loss + cross_entropy(ySmax, Tell);
109 if Tell(ym)==1
110 nCorrect = nCorrect + 1;
111 end
112 end
113 trainAccuracy(i) = nCorrect/N;
114 trainLoss(i) = loss/N;
115 subplot(211), plot(trainAccuracy(1:i), 'bo-'), hold on, grid on
116 ylim([0,1]), set(gca, 'FontSize', 12)
117 subplot(212), plot(trainLoss(1:i), 'bo-'), hold on, grid on
118 set(gca, 'FontSize', 12)
119 drawnow
120 %%%%%% 以上を入力 %%%%%%
121 %%%%%% 以下を入力 %%%%%%
122 if trainAccuracy(i)==1
123 break;
124 end
125 %%%%%% 以上を入力 %%%%%%
126 end
127 %%%%%% 以上を入力 %%%%%%
128 %% 入力データの描画
129 figure
130 for i=1:length(T)
131 if T(i)>0
132 p1 = plot(X(1, i), X(2, i), 'ro', 'MarkerFaceColor','r'); hold on
133 else
134 p2 = plot(X(1, i), X(2, i), 'b^', 'MarkerFaceColor','b'); hold on
135 end
136 end
137 axis([-0.5, 1.7, -0.5, 1.7]), grid on
138 xlabel('x1'), ylabel('x2')
139 % legend([p1, p2], クラス'+1', クラス'-1');
140 set(gca, 'FontSize', 14)
141 %%%%%% 以下を入力 %%%%%%
142 %% クラス分類の結果：推測したクラスの領域の描画2
143 [X1, X2] = meshgrid(-0.5:0.1:1.5);
144 for i=1:size(X1, 1)
145 for j=1:size(X1, 2)
146 Xell = [X1(i, j); X2(i, j)];
147 y = predict(Xell);
148 if y<0
149 plot(X1(i, j), X2(i, j), 'bx')
150 else
151 plot(X1(i, j), X2(i, j), 'r+')
152 end

```

```

153     end
154 end
155 legend([p1, p2], 'クラス+1', 'クラス-1');
156 %%%%% 以上を入力 %%%%
157 % ← (B)
158 %%%%% 以上を入力
159 %%%%% 以下を入力 %%%%
160 %% ソフトマックス関数出力
161 function y = predictSmax(X)
162 global W1 W2 W3 b1 b2 b3
163 global Z1 Z2 Z3
164 Z1 = sigmoid(W1*X + b1);
165 Z2 = sigmoid(W2*Z1 + b2);
166 Z3 = W3*Z2 + b3;
167 y = softmax(Z3);
168 end
169 %
170 %% ソフトマックス関数入力前でのクラス判定
171 function yc = predict(X)
172 global W1 W2 W3 b1 b2 b3
173 global Z1 Z2 Z3
174 Z1 = sigmoid(W1*X + b1);
175 Z2 = sigmoid(W2*Z1 + b2);
176 Z3 = W3*Z2 + b3;
177 [~, ind] = max(Z3);
178 if ind==1
179     yc = -1;
180 else
181     yc = 1;
182 end
183 end
184 %
185 %% シグモイド関数
186 function Y = sigmoid(Z)
187 Y = 1./(1 + exp(-Z));
188 end
189 %
190 %% ソフトマックス関数
191 function Y = softmax(Z)
192 c = max(Z);
193 exp_xc = exp(Z - c);
194 sum_exp_xc = sum(exp_xc);
195 Y = exp_xc/sum_exp_xc;
196 end
197 %
198 %% 一つのデータに対するクロスエントロピー誤差
199 function Lell = cross_entropy(Y, T)
200 c = 1.0e-7;
201 Lell = -sum(T.*log(Y + c));
202 end
203 %
204 %%%%% 以上を入力 %%%%
205 %%%%% 以下を入力 %%%%
206 %% 誤差逆伝搬法を用いた勾配の計算
207 function grad(X, Y, T)
208 global W2 W3
209 global dW1 dW2 dW3 db1 db2 db3
210 global Z1 Z2
211 % 勾配 db2, dW2 の計算
212 db3 = Y - T;
213 dW3 = db3*Z2';
214 qZ2 = Z2.*(1 - Z2);
215 db2 = W3'*db3.*qZ2;
216 dW2 = db2*Z1';

```

```

217 % 勾配 db1, dW1 の計算
218 qZ1 = Z1.*(1 - Z1);
219 db1 = W2'*db2.*qZ1;
220 dW1 = db1*X';
221 end

```

Listing 6.9: NN4l4layers

```

1 %%%%% 以下を入力
2 % プログラム3.2: NN2mlayer32.m
3 % 層の中間層を持つニューラルネットワークを用いた手書き数字認識2
4 %
5 clearvars, close all
6 %
7 global W1 W2 W3 b1 b2 b3
8 global dW1 dW2 dW3 db1 db2 db3
9 %
10 % データベースへのパスの設定Matlab
11 digitDatasetPath = fullfile(matlabroot, 'toolbox', 'nnet', 'nndemos', ...
12 'nndatasets', 'DigitDataset');
13 % データセットの設定
14 imds = imageDatastore(digitDatasetPath, ...
15 'IncludeSubfolders', true, 'LabelSource', 'foldernames');
16 % 個のサンプル画像を表示20
17 figure;
18 perm = randperm(10000, 20);
19 for i = 1:20
20     subplot(4, 5, i);
21     imshow(imds.Files{perm(i)});
22 end
23 drawnow
24 % 全体 (個) のデータの (個) を訓練データ, 残り (個) を10,00080%8,0002,000
25 % とテストデータする
26 N = 10000;
27 rateTrain = 0.8;
28 [imdsTrain, imdsTest] = splitEachLabel(imds, rateTrain, 'randomize');
29 %
30 dimInput = 784; % 入力の次元 () dimension
31 dimLayer1 = 50; % 中間層のノード数1
32 dimLayer2 = 100;
33 dimOutput = 10; % 出力の次元
34 rng(123) % 乱数のシード指定
35 % 重みとバイアスの初期設定. 乱数値を入力.
36 W1 = randn(dimLayer1, dimInput);
37 W2 = randn(dimLayer2, dimLayer1);
38 W3 = randn(dimOutput, dimLayer2);
39 b1 = randn(dimLayer1, 1);
40 b2 = randn(dimLayer2, 1);
41 b3 = randn(dimOutput, 1); % 重みとバイアスの初期設定
42 dW1 = zeros(size(W1));
43 dW2 = zeros(size(W2));
44 dW3 = zeros(size(W3));
45 db1 = zeros(size(b1));
46 db2 = zeros(size(b2));
47 db3 = zeros(size(b3));
48 % エポック数, ミニバッチのバッチサイズ, 学習率
49 numEpochs = 10;
50 batchSize = 100;
51 ep = 0.1;
52 %%%%% 以上を入力 %%%%
53 %%%%% 以下を入力 %%%%
54 %% 学習の準備と学習
55 % 反復回数, 分類精度, クロスエントロピー誤差の保存用
56 numTrain = N*rateTrain; % 訓練データの数

```

```

57 numTest = N - numTrain; % テストデータの数
58 numIters = numTrain/batchSize; % エポックにおけるミニバッチの反復数1
59 trainAccuracy = zeros(1, numEpochs);
60 trainLoss = zeros(1, numEpochs);
61 testAccuracy = zeros(1, numEpochs);
62 testLoss = zeros(1, numEpochs);
63 % 勾配の値を入れる変数の定義
64 gW1 = zeros(size(W1));
65 gW2 = zeros(size(W2));
66 gW3 = zeros(size(W3));
67 gb1 = zeros(size(b1));
68 gb2 = zeros(size(b2));
69 gb3 = zeros(size(b3));
70 % 学習
71 for i = 1:numEpochs
72     str = sprintf('エポック %d', i);
73     disp(str)
74     perm = randperm(numTrain);
75     for j = 1:numIters
76         for k = 1:batchSize
77             k1 = batchSize*(j - 1) + k;
78             img = readimage(imdsTrain, perm(k1));
79             tk1 = str2double(char(imdsTrain.Labels(perm(k1))));
80             Xk1 = double(reshape(img, [], 1));
81             Xk1 = Xk1/max(Xk1);
82             ySmax = predictSmax(Xk1);
83             Tk1 = zeros(10, 1); Tk1(tk1+1) = 1;
84             grad(Xk1, ySmax, Tk1);
85             gW1 = gW1 + dW1; gW2 = gW2 + dW2; gW3 = gW3 + dW3;
86             gb1 = gb1 + db1; gb2 = gb2 + db2; gb3 = gb3 + db3;
87         end
88         gW1 = gW1/batchSize; gW2 = gW2/batchSize; gW3 = gW3/batchSize;
89         gb1 = gb1/batchSize; gb2 = gb2/batchSize; gb3 = gb3/batchSize;
90         W1 = W1 - ep*gW1; W2 = W2 - ep*gW2; W3 = W3 - ep*gW3;
91         b1 = b1 - ep*gb1; b2 = b2 - ep*gb2; b3 = b3 - ep*gb3;
92         % 次の反復のための勾配の初期化
93         gW1 = gW1*0; gW2 = gW2*0; gW3 = gW3*0;
94         gb1 = gb1*0; gb2 = gb2*0; gb3 = gb3*0;
95     end
96
97     % 以下を入力
98     % 訓練データに対する精度
99     nCorrect = 0; loss = 0;
100    for j = 1:numTrain
101        img = readimage(imdsTrain, j);
102        tj = str2double(char(imdsTrain.Labels(j)));
103        Xj = double(reshape(img, [], 1));
104        Xj = Xj/max(Xj);
105        Tj = zeros(10,1); Tj(tj+1) = 1;
106        ySmax = predictSmax(Xj);
107        [~, ym] = max(ySmax);
108        loss = loss + cross_entropy(ySmax, Tj);
109        if ym==tj+1
110            nCorrect = nCorrect + 1;
111        end
112    end
113    trainAccuracy(i) = nCorrect/numTrain;
114    trainLoss(i) = loss/numTrain;
115
116    nCorrect = 0; loss = 0;
117    for j = 1:numTest
118        img = readimage(imdsTest, j);
119        tj = str2double(char(imdsTest.Labels(j)));
120        Xj = double(reshape(img, [], 1));

```

```

121 Xj = Xj/max(Xj);
122 Tj = zeros(10,1); Tj(tj+1) = 1;
123 ySmax = predictSmax(Xj);
124 [~, ym] = max(ySmax);
125 loss = loss + cross_entropy(ySmax, Tj);
126 if ym==tj+1
127     nCorrect = nCorrect + 1;
128 end
129 end
130 testAccuracy(i) = nCorrect/numTest;
131 testLoss(i) = loss/numTest;
132
133 subplot(211),
134 plot(trainAccuracy(1:i),'bo-'), hold on,
135 plot(testAccuracy(1:i),'r-'), hold on,
136 grid on
137 ylim([0,1]) , set(gca, 'FontSize', 12)
138 subplot(212),
139 plot(trainLoss(1:i),'bo-'), hold on,
140 plot(testLoss(1:i),'r-'), hold on,
141 grid on
142 set(gca, 'FontSize', 12)
143 drawnow
144 end
145 save trainRes10050e200 W1 W2 W3 b1 b2 b3 trainAccuracy trainLoss testAccuracy
    ↪ testLoss
146
147 %% ソフトマックス関数出力
148 function y = predictSmax(X)
149 global W1 W2 W3 b1 b2 b3
150 global Z1 Z2 Z3
151 Z1 = sigmoid(W1*X + b1);
152 Z2 = sigmoid(W2*Z1 + b2);
153 Z3 = W3*Z2 + b3;
154 y = softmax(Z3);
155 end
156
157 %
158 %% ソフトマックス関数入力前でのクラス判定
159 function yc = predict(X)
160 global W1 W2 W3 b1 b2 b3
161 global Z1 Z2 Z3
162 Z1 = sigmoid(W1*X + b1);
163 Z2 = sigmoid(W2*Z1 + b2);
164 Z3 = W3*Z2 + b3;
165 [~, ind] = max(Z3);
166
167 yc=ind-1;
168
169 end
170 %
171 %% シグモイド関数
172 function Y = sigmoid(Z)
173 Y = 1./(1 + exp(-Z));
174 end
175 %
176 %% ソフトマックス関数
177 function Y = softmax(Z)
178 c = max(Z);
179 exp_xc = exp(Z - c);
180 sum_exp_xc = sum(exp_xc);
181 Y = exp_xc/sum_exp_xc;
182 end
183 %

```



```

184 %% 一つのデータに対するクロスエントロピー誤差
185 function Le11 = cross_entropy(Y, T)
186 c = 1.0e-7;
187 Le11 = -sum(T.*log(Y + c));
188 end
189 %
190 % 以上を入力 %
191 % 以下を入力 %
192 %% 誤差逆伝搬法を用いた勾配の計算
193 function grad(X, Y, T)
194 global W2 W3
195 global dW1 dW2 dW3 db1 db2 db3
196 global Z1 Z2
197 % 勾配 db2, dW2 の計算
198 db3 = Y - T;
199 dW3 = db3*Z2';
200 qZ2 = Z2.*(1 - Z2);
201 db2 = W3'*db3.*qZ2;
202 dW2 = db2*Z1';
203 % 勾配 db1, dW1 の計算
204 qZ1 = Z1.*(1 - Z1);
205 db1 = W2'*db2.*qZ1;
206 dW1 = db1*X';
207 end
208 % 以上を入力 %

```